

HIRING TRACK / OBSERVABILITY PLATFORM ENGINEERING

Site Reliability Engineer: The Interview Learning Project

A complete, build-it-yourself preparation programme for the Platform SRE role. Eight weeks, one running capstone platform, and every concept the interview panel will probe — from SLO maths to telemetry pipeline economics.

Level 1 · Foundations

Level 2 · Operating the Platform

Level 3 · Platform Engineering at Scale

Capstone: Project Beacon

Interview Rehearsal

Read this first: what the job advert is actually asking for

Most candidates for this role fail for the same reason. They prepare as if it were an application-monitoring job. It is not. Read the advert again and notice the sentence that does the most work:

The disqualifying misread

"The focus of this role is the platform itself — its reliability, supportability, and usability." And then, explicitly: ownership of dashboards, alerts, or telemetry *for every application team* is not what is wanted.

You are being hired to run **the observability platform as a product**. Your customers are other engineering teams. Their applications are not your responsibility; the pipes, stores, dashboards-as-a-service, standards and self-service tooling they use *are*.

This distinction changes the answer to nearly every interview question. When they ask "how would you set an SLO?", a weak candidate describes an SLO for a checkout API. A strong candidate describes an SLO for *telemetry ingest availability* — and then explains how the platform helps three hundred teams define their own checkout SLOs without the platform team writing them.

Question	Application monitoring answer (weak)	Platform SRE answer (what they want)
What do you monitor?	Error rate and latency of the payments service.	Collector queue depth, spans dropped at the gateway, ingest-to-queryable lag, query p99, per-tenant quota rejection rate.
Who is your customer?	The business.	Engineering teams onboarding to the platform, plus the incident-management function that depends on our alert delivery path.
How do you reduce alert noise?	Tune thresholds on the noisy alerts.	Publish an alerting standard and a linter, measure alert precision per tenant, expose a self-service alert template, and report an alert-quality score back to teams.
A team's dashboard is wrong. What do you do?	Fix their dashboard.	Determine whether the platform mis-ingested the data (my defect) or the team mis-instrumented (their defect, my documentation gap). Fix the class of problem, not the instance.
How do you scale your team's work?	Hire more engineers.	Golden paths: a tenant onboards by raising a pull request against a manifest; the pipeline provisions routing, quotas, retention and a starter dashboard with no human in the loop.

How to use this document

Every level ends with something you have actually built. The three levels feed one capstone — **Project Beacon**, a miniature multi-tenant observability platform you run on your own machine. By the interview you should be able to open a laptop, show a repository, and walk through a real incident you caused and resolved on it. That is worth more than any certificate.

Level 1 • Foundations

1. The vocabulary, defined properly
2. The architecture you will build
3. Environment setup
4. First telemetry: C# and Python
5. Linux and networking triage
6. Level 1 cheatsheet

Level 2 • Operating the Platform

1. Telemetry pipeline architecture
2. Meta-monitoring
3. Platform SLIs, SLOs, error budgets
4. ClickHouse for telemetry
5. Alert quality and correlation
6. Incident and problem management
7. Runbooks that work at 3am
8. Level 2 cheatsheet

Level 3 • Platform at Scale

1. GitOps and environment promotion
2. Self-service and golden paths
3. Telemetry economics
4. Multi-tenancy and isolation
5. Resilience and service readiness
6. Regulated environments
7. Enablement and documentation

Also in this guide

[Project Beacon](#) — the eight-week capstone plan · [Interview rehearsal: 45 questions, design exercises, troubleshooting drills and STAR stories](#) · [Career roadmap and certifications](#)

Level 1 • Foundations

The language, the mental model and a running stack on your laptop. Target: two weeks. You finish this level able to explain observability to a non-specialist and to trace a single request from a C# controller to a Grafana panel.

1. The vocabulary, defined properly

Interviewers use these terms as shibboleths. Getting [SLA](#) and [SLO](#) the wrong way round, or describing observability as "three pillars and nothing else", signals that you have read a blog post rather than run a platform. Learn these until they are boring.

Reliability language

Site Reliability Engineering (SRE)

An operations discipline that applies software engineering to infrastructure and operations problems, with the explicit goal of replacing manual work with automation. Originated at Google; the defining texts are freely readable online. The core bargain: reliability is a feature with a cost, so you decide how much you want, measure it, and spend the remainder on change.

Toil

Manual, repetitive, automatable, tactical work that scales linearly with the size of the service and produces no enduring value. Restarting a stuck collector by hand is toil. Writing the controller that restarts it is engineering. Google's rule of thumb caps toil at 50% of an SRE's time; the interview question is usually "give me an example of toil you removed and what you measured".

SLI — Service Level Indicator

A carefully defined quantitative measure of some aspect of service quality, expressed as a ratio of good events to valid events. Not "CPU is at 60%". Rather: [proportion of OTLP export requests returning 2xx within 5s](#). A good SLI is measured as close to the user as possible and is expressed as a percentage between 0 and 100.

SLO — Service Level Objective

A target value or range for an SLI over a stated compliance window. `99.9% of OTLP export requests succeed within 5s, measured over a rolling 28 days`. Internal. Chosen deliberately, not aspirationally — an SLO you never breach is set too low to be useful, because it never forces a trade-off conversation.

SLA — Service Level Agreement

A contract with a customer that carries a financial or contractual consequence when breached. SLAs are always looser than the internal SLO that protects them. If your SLA says 99.5%, your SLO should say 99.9%, so you find out you are in trouble long before the lawyers do. In an internal platform team the "SLA" may be a formal service agreement with the business rather than an external contract — say so, it shows you understand context.

Error budget

The permitted unreliability: `100% - SLO`, converted into a quantity of bad events or minutes over the window. A 99.9% SLO over 28 days allows roughly 40 minutes of total badness. The budget is the currency in which reliability and delivery speed are traded. Burn it slowly and you may ship freely; burn it fast and you freeze changes and fix the platform. The *policy* attached to the budget matters more than the number.

SLO	Allowed bad time per 28 days	Per 30 days	Per year	Realistic for
99%	6h 43m	7h 12m	3d 15h	Non-production ingest, internal batch tooling
99.5%	3h 21m	3h 36m	1d 19h	Query/read path for exploratory analysis
99.9%	40m	43m	8h 45m	Telemetry ingest, dashboard availability
99.95%	20m	21m	4h 22m	Alert evaluation and delivery path
99.99%	4m	4m 20s	52m	Rarely justified for an internal platform — say so out loud

Interview gold

When asked what SLO you would set, the strongest possible answer is: "I would not pick a number first. I would measure the current SLI for four weeks, show the teams what they are already getting, ask what actually hurts them when it degrades, and set the SLO just above the level that causes real pain. Then I would attach a policy — what happens when the budget is exhausted — because an SLO with no consequence is a dashboard decoration."

Monitoring vs observability

Monitoring tells you whether a system is in a state you predicted in advance — you defined the check, the threshold and the dashboard. **Observability** is the property of being able to ask new questions about a system's internal state from its external outputs, without shipping new code to answer them. Monitoring answers "is it broken?"; observability answers "why is it broken in this way I have never seen before?". High-cardinality, high-dimensionality data is what makes the second possible.

The signals

Metrics — numeric measurements aggregated over time. Cheap, dense, poor at explaining a single request.

Logs — discrete timestamped events, ideally structured. Expensive at volume, excellent detail.

Traces — a causally linked tree of spans describing one request as it crosses service boundaries. The only signal that explains *where* time went in a distributed call.

Profiles — continuous CPU/memory sampling attributable to code paths; OpenTelemetry's fourth signal, which reached release-candidate status in early 2026.

Do not recite "three pillars" as if it were a law. The useful framing is that these are different *encodings* of the same events, and the platform's real job is to let a user pivot between them on shared context — usually `trace_id` and the resource attributes.

Cardinality

The number of distinct time series produced by a metric, equal to the product of the distinct values of each of its labels. `http_requests_total{method, status, endpoint}` with 5 methods × 20 statuses × 500 endpoints is 50,000 series. Add `user_id` and you have destroyed your metrics store. Cardinality control is one of the top three recurring operational problems on any telemetry platform, and being able to describe how you would detect and stop a cardinality explosion is a near-guaranteed interview question.

Semantic conventions

OpenTelemetry's specification of standard attribute names and units, so that `http.request.method` means the same thing regardless of which language, library or vendor produced it. This is what makes cross-team dashboards possible without per-team configuration, and it is exactly the "shared standards" the advert asks you to help teams adopt. Attributes carry a stability level (Development, Release Candidate, Stable) — pin dashboards to stable ones.

Golden signals • RED • USE

Four golden signals (Google): latency, traffic, errors, saturation.

RED (Tom Wilkie, request-scoped services): Rate, Errors, Duration.

USE (Brendan Gregg, resources): Utilisation, Saturation, Errors.

Use RED for the request-handling components of your platform (the OTLP receiver, the query API) and USE for the resources underneath (disk, CPU, connection pools, collector queues). Saying which you would apply where — and why — reads as experience.

Operations language

Incident vs problem

An **incident** is an unplanned interruption or reduction in quality; the goal is to restore service as fast as possible, and workarounds are acceptable. A **problem** is the underlying cause of one or more incidents; the goal is permanent elimination, and speed is not the priority. Confusing the two is why organisations resolve the same incident eleven times. The advert's "problem management and root cause analysis" plus "drive follow-up actions that reduce repeat issues" is asking directly about this loop.

MTTD • MTTA • MTTR • MTBF

Mean time to **detect**, to **acknowledge**, to **restore** (or repair/resolve — say which you mean), and mean time **between** failures. Break MTTR down in interviews: detection, acknowledgement, diagnosis, mitigation, verification. Most platform teams that "have an MTTR problem" actually have a diagnosis problem, which is an observability problem, which is your product.

Blameless postmortem

A written retrospective that treats human error as a symptom of system design rather than a cause, on the premise that people act reasonably given the information and tooling available at the time. Blameless does not mean accountability-free: the actions are owned, dated and tracked. If you have never written one, write one for a self-inflicted failure in your capstone — it is a superb interview artefact.

Service readiness / production readiness review

A gate before a service (or a new platform tenant) is allowed into production: does it have an owner, an SLO, dashboards, alerts routed to a real rota, a runbook, a capacity estimate, a rollback path, and a dependency map? The advert lists "contribute to service readiness through dashboards, SLOs, SLIs, runbooks, resilience checks and performance validation" — that sentence is describing a readiness checklist. Bring one.

GitOps

An operating model where the desired state of a system is declared in Git, and an automated controller continuously reconciles the running state to match. Four principles: the state is *declarative*, it is *versioned and immutable*, it is *pulled automatically*, and it is *continuously reconciled*. The consequences that matter operationally: your change history is your audit log, rollback is `git revert`, and configuration **drift** — a running state that no longer matches the declared state — becomes detectable and correctable rather than invisible.

Golden path / paved road

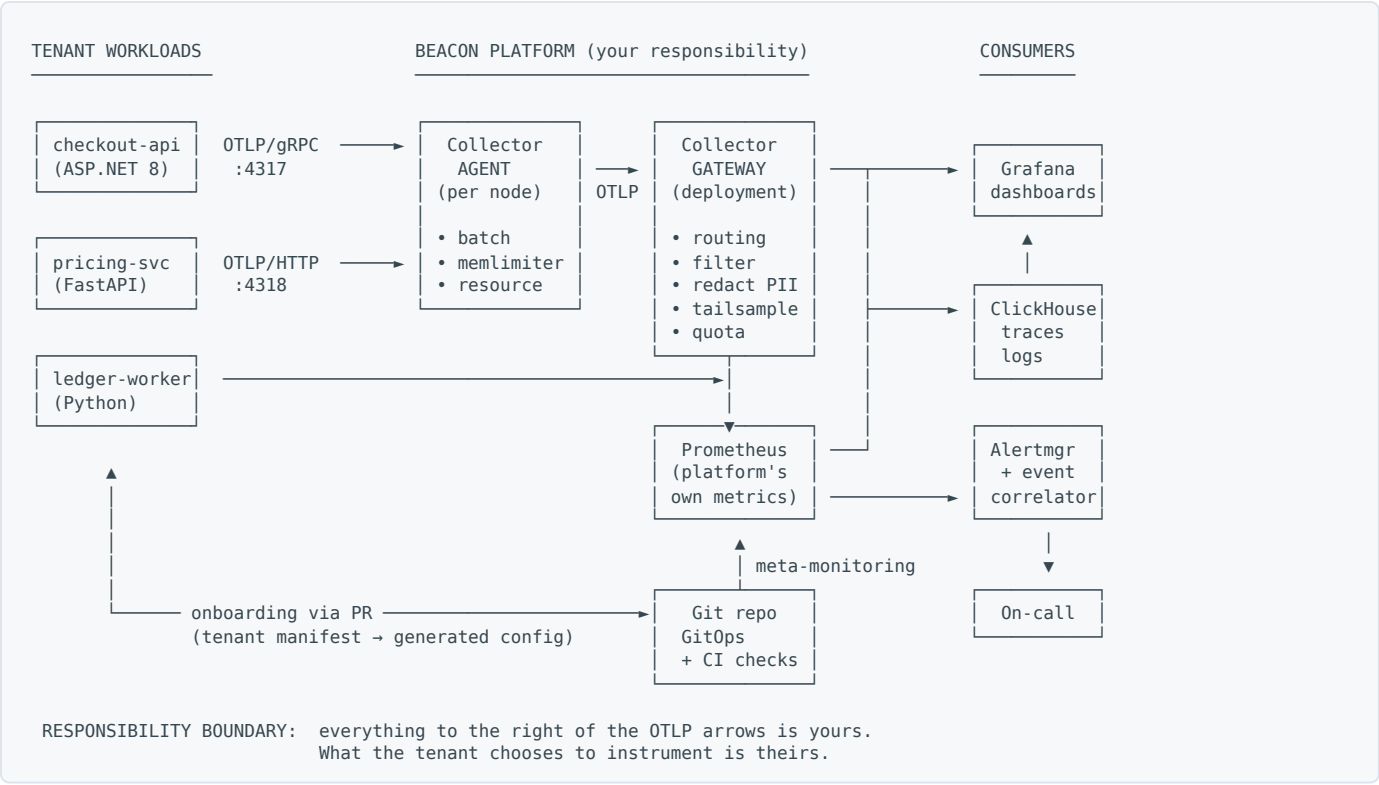
The supported, opinionated, well-documented way to do a common thing on your platform, made so much easier than the alternatives that teams choose it voluntarily. A golden path is not a mandate; it is a bribe. "Self-service tooling so teams can operate with less reliance on direct support" is the advert asking whether you can build one.

Tenant • multi-tenancy • noisy neighbour

On a shared platform, a **tenant** is a team or service with its own data, quotas and access. **Noisy neighbour** is the failure mode where one tenant's behaviour — a cardinality explosion, a log flood after a bad deploy, a runaway query — degrades everyone else. Nearly every hard problem on an internal platform is a noisy-neighbour problem in disguise.

2. The architecture you will build

Project Beacon is a deliberately small but structurally honest version of the platform this role operates. Every component maps to something named in the advert or its desirable list.



Beacon component	Stands in for	Advert line it satisfies
OpenTelemetry Collector (agent + gateway)	Cribl / Grafana Alloy / vendor agents	"telemetry pipelines, data routing, data filtering, retention"
ClickHouse	ClickHouse (named explicitly)	"observability, monitoring, telemetry or data pipeline technologies"
Grafana + provisioned dashboards	Grafana (named explicitly)	"contribute to service readiness through dashboards"
Prometheus scraping the collectors	Meta-monitoring of any platform	"monitor platform health, investigate and diagnose issues"
Alertmanager + a small correlation service	BigPanda-style event correlation	"alert quality measures, service health models"
Redis	Redis (named explicitly) — dedup/rate-limit state	"Redis, Flink, or similar tools"
Git repo + CI + Argo CD (or Flux)	GitOps workflow	"GitOps workflows... pull requests, environment promotion, configuration-as-code"
Tenant manifest + generator	Self-service onboarding	"self-service tooling so teams can operate with less reliance on direct support"

On the named tools

You are not expected to have used Cribl, BigPanda or Flink. They are listed as desirable. What you must be able to do is explain *what class of problem each solves* and demonstrate the equivalent capability in something you have actually run. "I have not used Cribl, but I have built the same routing-and-reduction stage with the OpenTelemetry Collector's filter, transform and routing processors, and here is the cost reduction I measured" is a much stronger answer than a memorised product description.

3. Environment setup

Two environments. Docker Compose for fast iteration on the pipeline; a `kind` Kubernetes cluster for the GitOps and multi-tenancy work in Level 3. Do Compose first.

1 Prerequisites

Docker Desktop or Docker Engine with Compose v2, Git, .NET 8 SDK, Python 3.11+, and `kubect1`. On Windows, work inside WSL2 — the Linux muscle memory is part of what you are being assessed on, and the advert asks for "cloud, container, Linux, networking or distributed system" understanding.

2 Create the repository with the shape you will defend in the interview

Repository layout is itself an answer to "how do you manage configuration as code". Do not let it grow organically.

3 Bring up the stack and confirm data flows end to end

Do not move on until a trace emitted by the C# service is visible in Grafana, queried from ClickHouse, with its `service.name` and `trace_id` intact.

repository layout – beacon/

```
# Structure it as a product from day one. Interviewers read repos.
beacon/
├── README.md                # what Beacon is, who it serves, how to onboard
├── docs/
│   ├── onboarding.md        # the self-service guide for tenant teams
│   ├── telemetry-standards.md # naming, attributes, what we reject and why
│   ├── slo-catalogue.md     # the platform's own SLOs and their rationale
│   └── runbooks/
│       ├── ingest-backlog.md
│       ├── clickhouse-disk.md
│       └── cardinality-spike.md
├── platform/
│   ├── collector-agent.yaml
│   ├── collector-gateway.yaml
│   ├── clickhouse/ddl/
│   ├── grafana/dashboards/    # provisioned as JSON, never hand-edited in the UI
│   ├── grafana/provisioning/
│   └── prometheus/rules/      # meta-monitoring + burn-rate alerts
├── tenants/
│   ├── _template.yaml        # the golden path
│   └── checkout-api.yaml
```

```

├── pricing-svc.yaml
├── tools/
│   ├── generate-config.py      # tenant manifests → collector config
│   └── validate-standards.py   # CI gate on telemetry standards
├── workloads/
│   ├── checkout-api/          # ASP.NET Core 8
│   └── pricing-svc/           # FastAPI
├── .github/workflows/ci.yaml
└── compose.yaml

```

compose.yaml (abridged – the shape matters more than the versions)

```

services:
  clickhouse:
    image: clickhouse/clickhouse-server:latest
    ports: ["8123:8123", "9000:9000"]
    ulimits: { nofile: { soft: 262144, hard: 262144 } }
    volumes:
      - ./platform/clickhouse/ddl:/docker-entrypoint-initdb.d:ro
      - ch-data:/var/lib/clickhouse
    healthcheck:
      test: ["CMD", "wget", "--spider", "-q", "localhost:8123/ping"]
      interval: 10s

  otel-gateway:
    image: otel/opentelemetry-collector-contrib:latest # contrib: you need the extra processors
    command: ["--config=/etc/otel/gateway.yaml"]
    volumes: ["./platform/collector-gateway.yaml:/etc/otel/gateway.yaml:ro"]
    ports:
      - "4317:4317" # OTLP gRPC
      - "4318:4318" # OTLP HTTP
      - "8888:8888" # the collector's OWN metrics – this is your meta-monitoring
      - "13133:13133" # health_check extension
    depends_on:
      clickhouse: { condition: service_healthy }

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./platform/prometheus:/etc/prometheus:ro
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana:latest
    environment:
      GF_INSTALL_PLUGINS: grafana-clickhouse-datasource
      GF_AUTH_ANONYMOUS_ENABLED: "true" # lab only – never in a regulated environment
    volumes:
      - ./platform/grafana/provisioning:/etc/grafana/provisioning:ro
      - ./platform/grafana/dashboards:/var/lib/grafana/dashboards:ro
    ports: ["3000:3000"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

volumes: { ch-data: {} }

```

Pin your versions before the interview

`:latest` is fine while you are learning and indefensible in the conversation. Before you demo, pin every image to a digest or explicit tag and be ready to explain your upgrade process. "How do you upgrade a collector fleet without dropping telemetry?" is a very common question — the answer involves rolling updates, the gateway's persistent queue, agent-side retry, and a canary tenant.

4. First telemetry: C# and Python

You will write tenant workloads yourself so you understand what your users experience. Instrumenting badly on purpose — and then catching it with your own standards linter — is one of the best stories you can bring to the interview.

ASP.NET Core 8 tenant service

workloads/checkout-api/Program.cs

```
using OpenTelemetry.Logs;
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;
using System.Diagnostics;
using System.Diagnostics.Metrics;

var builder = WebApplication.CreateBuilder(args);

// One ActivitySource per service, named after the service. This name shows up as the
// instrumentation scope in every span — the platform uses it to attribute custom spans.
var activitySource = new ActivitySource("Checkout.Api");
var meter = new Meter("Checkout.Api");
var basketValue = meter.CreateHistogram<double>(
    "checkout.basket.value", unit: "GBP", description: "Value of submitted baskets");

// Resource attributes identify WHO is emitting. The platform's admission control keys on these.
// service.name is mandatory; service.namespace and deployment.environment.name are how a
// multi-tenant platform routes, meters and bills.
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r
        .AddService(serviceName: "checkout-api", serviceVersion: "1.4.2")
        .AddAttributes(new Dictionary<string, object>
        {
            ["service.namespace"] = "retail",
            ["deployment.environment.name"] = builder.Environment.EnvironmentName.ToLower(),
            ["beacon.tenant"] = "retail-checkout", // platform-specific: quota key
            ["beacon.cost centre"] = "CC-4471" // enables showback
        })
    .WithTracing(t => t
        .AddSource(activitySource.Name)
        .AddAspNetCoreInstrumentation(o =>
        {
            // Do not trace your own health probes. This is the single easiest volume win
            // and the first thing you should teach tenant teams.
            o.Filter = ctx => !ctx.Request.Path.StartsWithSegments("/healthz");
            o.RecordException = true;
        })
        .AddHttpClientInstrumentation())
```

```

        .AddOtlpExporter()
    .WithMetrics(m => m
        .AddMeter(meter.Name)
        .AddAspNetCoreInstrumentation()
        .AddRuntimeInstrumentation()
        .AddOtlpExporter())
    .WithLogging(l => l.AddOtlpExporter());

var app = builder.Build();

app.MapGet("/healthz", () => Results.Ok("ok"));

app.MapPost("/checkout", async (BasketRequest req, IHttpContextFactory http) =>
{
    // A manual span for the business operation. Note the attribute naming: dots, lower
    // case, namespaced by domain. This is what your standards document will mandate.
    using var activity = activitySource.StartActivity("checkout.submit", ActivityKind.Internal);
    activity?.SetTag("checkout.item.count", req.Items.Count);
    activity?.SetTag("checkout.currency", "GBP");
    // NEVER: activity?.SetTag("customer.email", req.Email);
    // High cardinality AND personal data. The gateway redacts it, but the fix belongs here.

    basketValue.Record(req.Total, new KeyValuePair<string, object?>("checkout.channel", req.Channel));

    try
    {
        var client = http.CreateClient();
        var price = await client.GetStringAsync("http://pricing-svc:8000/price");
        activity?.SetStatus(ActivityStatusCode.Ok);
        return Results.Ok(new { req.Total, price });
    }
    catch (Exception ex)
    {
        // Recording the exception on the span is what turns a trace into a diagnosis.
        activity?.AddException(ex);
        activity?.SetStatus(ActivityStatusCode.Error, ex.Message);
        throw;
    }
});

app.Run();

record BasketRequest(List<string> Items, double Total, string Channel, string Email);

```

workloads/checkout-api/appsettings.json – configuration by environment variable in production

```

# The SDK reads these automatically. In Kubernetes they come from a ConfigMap that
# the platform injects – the tenant never hand-writes an endpoint.
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-agent:4317
OTEL_EXPORTER_OTLP_PROTOCOL=grpc
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1
OTEL_METRIC_EXPORT_INTERVAL=30000
OTEL_RESOURCE_ATTRIBUTES=service.namespace=retail,beacon.tenant=retail-checkout

```

Python (FastAPI) tenant service

workloads/pricing-svc/main.py

```

# pip install opentelemetry-distro opentelemetry-exporter-otlp \
#           opentelemetry-instrumentation-fastapi opentelemetry-instrumentation-redis
# then run with: opentelemetry-instrument uvicorn main:app --host 0.0.0.0 --port 8000
# Zero-code instrumentation first, manual spans only where the business logic needs them.

```

```

import os, random, time
from fastapi import FastAPI, HTTPException
from opentelemetry import trace, metrics

app = FastAPI()
tracer = trace.get_tracer("pricing.svc")
meter = metrics.get_meter("pricing.svc")

# A histogram, not a gauge: you cannot compute a percentile from a gauge.
lookup_duration = meter.create_histogram(
    "pricing.lookup.duration", unit="s", description="Price lookup latency")
cache_ops = meter.create_counter("pricing.cache.operations")

@app.get("/healthz")
def healthz():
    return {"status": "ok"}

@app.get("/price")
def price(sku: str = "SKU-1"):
    start = time.perf_counter()
    with tracer.start_as_current_span("pricing.lookup") as span:
        # Bound the cardinality yourself. sku has ~50k values; a bucketed
        # attribute keeps the metric usable while the span keeps the detail.
        span.set_attribute("pricing.sku", sku) # fine on a span
        cache_ops.add(1, {"result": "hit"}) # fine on a metric: 2 values
        # cache_ops.add(1, {"sku": sku}) <-- 50,000 series. Never.

        if random.random() < 0.02:
            span.set_status(trace.StatusCode.ERROR, "upstream price feed timeout")
            raise HTTPException(status_code=503, detail="price feed unavailable")

    time.sleep(random.gauss(0.04, 0.02))
    lookup_duration.record(time.perf_counter() - start, {"channel": "web"})
    return {"sku": sku, "price": 19.99}

```

Do this deliberately

Add the `customer.email` tag and the `sku` metric label on purpose, watch what happens to your series count in Prometheus and your row count in ClickHouse, then build the guard rails that stop it. You will then have a first-hand answer to "tell me about a time you dealt with a cardinality problem" — and first-hand beats theoretical every time.

Level 1 references

- [Google SRE Book](#) — free online. Read chapters 1–6 and the postmortem chapter before any interview.
- [The SRE Workbook](#) — the practical companion. Chapters 2 (implementing SLOs) and 5 (alerting on SLOs) are the ones you will be quizzed on.
- [OpenTelemetry documentation](#) — start with Concepts, then Collector, then your language.
- [OpenTelemetry .NET and OpenTelemetry Python](#)
- [Microsoft: .NET observability with OpenTelemetry](#) — the idiomatic ASP.NET Core path.
- [Semantic conventions specification](#) — the source of truth for attribute naming. Bookmark it; you will quote it in the interview.
- [OpenTelemetry Demo \(Astronomy Shop\)](#) — a full polyglot microservice system that emits real telemetry. Excellent load generator for your pipeline.
- [Prometheus metric and label naming](#)
- [Brendan Gregg: the USE method](#)

5. Linux and networking triage

"Solid understanding of cloud, container, Linux, networking or distributed system environments" is a live-troubleshooting screen in disguise. Expect to be shown a symptom and asked what you would run. Know these cold and, more importantly, know what each one *tells* you.

Symptom	What you run	What you are looking for
Collector pod restarting	<code>kubectl describe pod</code> , <code>kubectl logs --previous</code> , <code>dmesg -T grep -i oom</code>	OOMKilled with exit 137 → <code>memory_limiter</code> mis-set or a queue growing unbounded. Restart reason beats log spelunking.
Telemetry arriving late	<code>curl -s localhost:8888/metrics grep queue</code>	<code>otelcol_exporter_queue_size</code> approaching <code>queue_capacity</code> means the exporter cannot drain — the fault is downstream, not in the collector.
Exports failing intermittently	<code>ss -tanp state established</code> , <code>ss -s</code> , <code>conntrack -S</code>	Ephemeral port or conntrack table exhaustion; <code>TIME_WAIT</code> accumulation from a client that will not reuse connections.
"It's the network"	<code>dig +short</code> , <code>mtr -rw</code> , <code>ss -tn</code> , <code>curl -w '%{time_connect} % {time_starttls} % {time_total}\n'</code>	Split the latency: DNS vs TCP connect vs TLS handshake vs first byte. Naming the stage is what separates an engineer from a guesser.
Disk filling on the store node	<code>df -h</code> , <code>du -xh --max-depth=1 /var/lib sort -h</code> , <code>lsof +L1</code>	Deleted-but-open files hold space until the process closes them — a classic that <code>du</code> alone will never show you.
High CPU, unclear cause	<code>top -H -p PID</code> , <code>pidstat 1</code> , <code>perf top</code>	Which <i>thread</i> . Then whether it is user, system or iowait time — three completely different investigations.

Symptom	What you run	What you are looking for
Slow queries on the store	<code>iostat -x 1</code> , <code>vmstat 1</code>	<code>%util</code> near 100 with rising <code>await</code> means saturated storage; high <code>si/so</code> in <code>vmstat</code> means you are swapping and everything else is a red herring.
Container cannot reach a service	<code>kubectl exec -- nslookup svc</code> , <code>kubectl get endpoints</code> , <code>netstat -rn</code>	Empty endpoints means the selector or the readiness probe is the problem, not the network.

The framing that impresses

Do not answer troubleshooting questions with a list of commands. Answer with a *strategy*, then the commands: "First I establish blast radius — one tenant or all? Then I check whether the platform's own health signals degraded before or after the symptom, because that tells me cause versus consequence. Then I bisect the pipeline: is data arriving at the agent, leaving the gateway, landing in the store, or failing at query? Each stage has one metric that answers it, and here they are."

6. Level 1 cheatsheet

Command	Purpose
<code>docker compose up -d --build</code>	Bring the whole Beacon stack up
<code>docker compose logs -f otel-gateway</code>	Follow collector logs; most config errors surface here at start-up
<code>curl -s localhost:13133</code>	Collector health_check extension — your liveness signal
<code>curl -s localhost:8888/metrics grep otelcol_</code>	The collector's own telemetry: the heart of meta-monitoring
<code>otelcol-contrib validate --config=gateway.yaml</code>	Validate collector config — put this in CI as a required check
<code>curl -X POST localhost:4318/v1/traces -H 'Content-Type: application/json' -d @span.json</code>	Hand-inject a span; the fastest way to test a pipeline change in isolation
<code>clickhouse-client -q "SELECT count() FROM otel.otel_traces"</code>	Confirm data actually landed
<code>kubectl get pods -A --sort-by=.status.containerStatuses[0].restartCount</code>	Find the thing that is quietly crash-looping
<code>kubectl top pod -n beacon</code>	Live resource usage against requests/limits
<code>promtool check rules rules.yaml</code>	Validate Prometheus alerting rules — another required CI check
<code>promtool test rules tests.yaml</code>	Unit-test your alerts. Almost nobody does this; mention it and watch the interviewer sit up.

Level 1 done when...

You can delete the whole stack, re-create it from the repository with one command, generate load, and point at a Grafana panel showing p99 latency for `checkout-api` sourced from ClickHouse — and explain every hop the data took to get there.

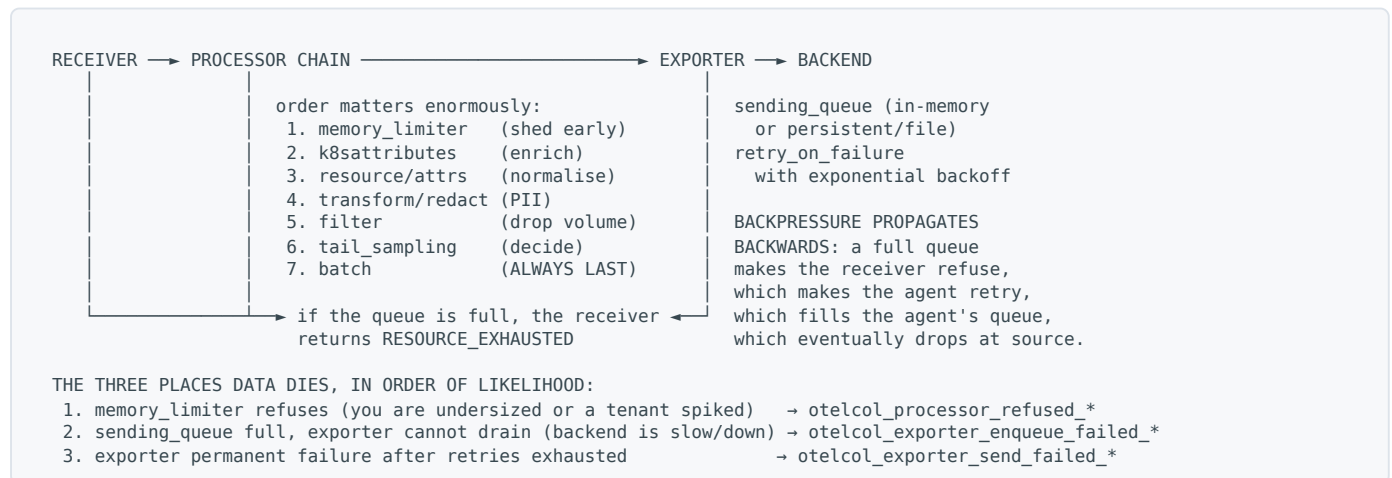


Level 2 • Operating the Platform

Where the interview is won. Pipeline internals, meta-monitoring, SLO mathematics, alert quality and the incident lifecycle. Target: three weeks. You finish this level able to run a game day on your own platform and write the postmortem.

7. Telemetry pipeline architecture

The collector is not "the thing that forwards data". It is a queueing system with admission control, and every serious platform incident you will handle is a queueing problem. Understand the internal path and you can diagnose almost anything.



Agent vs gateway, and why you want both

Agent (DaemonSet / sidecar)

- Runs next to the workload; receives over localhost so a network blip does not lose data
- Enriches with host and Kubernetes metadata that only exists locally
- Cheap batching to reduce connection count against the gateway
- Must stay light: it competes with the tenant's application for node resources, and starving a tenant to collect telemetry is an unforgivable platform sin

Gateway (horizontally scaled Deployment)

- The policy enforcement point: quotas, redaction, filtering, routing, sampling
- Central place to change behaviour for all tenants without touching workloads
- Holds the persistent queue that absorbs backend outages
- Where tail sampling must live, because it needs all spans of a trace — which forces `loadbalancing` routing by `trace_id` when you have more than one gateway replica

platform/collector-gateway.yaml — the config you will be asked to explain line by line

```
receivers:
  otlp:
    protocols:
      grpc: { endpoint: "0.0.0.0:4317", max_recv_msg_size_mib: 16 }
      http: { endpoint: "0.0.0.0:4318" }

processors:
  # FIRST. Always. It sheds load before anything else allocates memory.
  # limit_percentage is preferred in containers: it reads the cgroup limit.
  memory_limiter:
    check_interval: 1s
    limit_percentage: 75
    spike_limit_percentage: 20

  # Normalise. Tenants get this wrong; the platform makes it right exactly once.
  resource:
    attributes:
      - { key: beacon.ingest.gateway, value: "gw-eu-west-1", action: insert }
      - { key: deployment.environment, action: delete } # deprecated name

  # PII redaction. In a regulated environment this is not optional and it is
  # YOUR control, because you cannot trust 300 teams to never log a card number.
  transform/redact:
    error_mode: ignore
    trace_statements:
      - 'delete_key(span.attributes, "customer.email")'
      - 'delete_key(span.attributes, "http.request.header.authorization")'
      - 'replace_pattern(span.attributes["db.query.text"], "[0-9]{13,19}", "[REDACTED-PAN]")'
    log_statements:
      - 'replace_pattern(log.body, "[\\w.]+@[\\w.]+", "[REDACTED-EMAIL]")'

  # Volume reduction. Health checks are ~30% of spans in a typical estate and
  # have never once helped anyone during an incident.
  filter/noise:
    error_mode: ignore
```

```

traces:
  span:
    - 'attributes["http.route"] == "/healthz"'
    - 'attributes["url.path"] == "/metrics"'
  logs:
    log_record:
      - 'severity_number < SEVERITY_NUMBER_INFO and resource.attributes["deployment.environment.name"] == "dev"'

# Keep what is interesting, drop what is not. Tail sampling needs the WHOLE trace,
# so decision_wait must exceed your slowest realistic request.
tail_sampling:
  decision_wait: 10s
  num_traces: 100000
  policies:
    - { name: keep-all-errors, type: status_code, status_code: { status_codes: [ERROR] } }
    - { name: keep-slow, type: latency, latency: { threshold_ms: 750 } }
    - { name: keep-vip-tenant, type: string_attribute,
        string_attribute: { key: beacon.tenant, values: [payments-core] } }
    - { name: baseline, type: probabilistic, probabilistic: { sampling_percentage: 5 } }

# LAST. Batching before filtering wastes work; batching before the limiter hides pressure.
batch:
  timeout: 5s
  send_batch_size: 8192
  send_batch_max_size: 10000

exporters:
  clickhouse:
    endpoint: tcp://clickhouse:9000?dial_timeout=10s
    database: otel
    ttl: 720h
    sending_queue:
      enabled: true
      num_consumers: 10
      queue_size: 10000
      storage: file_storage # survives a restart – the difference between
    retry_on_failure: # a blip and a data-loss incident
      enabled: true
      initial_interval: 5s
      max_elapsed_time: 300s

extensions:
  health_check: { endpoint: "0.0.0.0:13133" }
  file_storage: { directory: /var/lib/otelcol/queue }
  pprof: { endpoint: "0.0.0.0:1777" } # when the collector itself misbehaves

service:
  extensions: [health_check, file_storage, pprof]
  telemetry:
    metrics:
      level: detailed # normal/detailed changes which self-metrics you get
      logs: { level: warn }
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, transform/redact, filter/noise, tail_sampling, batch]
      exporters: [clickhouse]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, resource, transform/redact, filter/noise, batch]
      exporters: [clickhouse]

```

Three questions you will be asked about this file

Why is `memory_limiter` first? Because shedding load is only useful before you have spent memory on the data. Placed later it protects nothing.

Why is `batch` last? Because every processor before it operates more efficiently on batches it did not have to rebuild, and because batching first means filtering and sampling do work on data you were about to discard.

What breaks tail sampling when you scale the gateway to five replicas? Spans from one trace land on different replicas, so no replica sees the whole trace and the policy decides on partial data. Fix: a front tier using the `loadbalancing` exporter with `routing_key: traceID` so all spans of a trace reach the same sampling replica.

8. Meta-monitoring: who watches the watchmen

This is the single most role-specific topic in the whole guide. If the observability platform is down, nobody can see that the observability platform is down. Your answer to this establishes immediately whether you have run one.

Meta-monitoring

Monitoring of the monitoring system itself, deliberately built so that it does not depend on the system it is monitoring. In practice: a small, separate, boring Prometheus (or a second region, or a managed vendor account) scrapes the collectors, the store and the query layer, and alerts through a path that does not traverse the primary pipeline. Independence is the whole point — if your alert about the pipeline is delivered *by* the pipeline, you have a circular dependency that will fail exactly when you need it.

Signal	What it tells you	Alert on
<code>otelcol_receiver_accepted_spans</code> vs <code>otelcol_receiver_refused_spans</code>	Whether you are admitting or shedding at the front door	Refused ratio > 0.1%
<code>otelcol_processor_dropped_spans</code>	Data destroyed inside the pipeline — always a defect or a deliberate policy you should be able to name	Any non-policy drop
<code>otelcol_exporter_queue_size</code> / <code>otelcol_exporter_queue_capacity</code>	Headroom before you start refusing. The leading indicator of every ingest incident.	> 60% for 5m
<code>otelcol_exporter_send_failed_spans</code>	Backend rejecting or unreachable. Recent collector releases attach richer failure attributes so you can classify the cause rather than guess.	> 0 sustained 2m
<code>otelcol_process_memory_rss</code> against the cgroup limit	How close you are to an OOMKill	> 80% for 10m
Ingest lag: <code>now() - max(span.end_time)</code> in the store	End-to-end freshness. The number your users actually feel.	> 60s

Signal	What it tells you	Alert on
Synthetic canary trace round-trip	True end-to-end: emit a known span every 30s, query for it, measure the delay	Missing for 3 consecutive probes
Grafana datasource query p99	The read path, which users blame first and you check last	> 5s
ClickHouse parts per partition, merge queue depth	Too-frequent small inserts; the classic cause of a store that slowly stops accepting writes	Parts > 300

Build the canary. It is the best five hours you will spend.

A tiny service that emits a span with a known unique id, waits, then queries ClickHouse for it and records `beacon.canary.roundtrip.seconds` plus a success counter. This one component gives you a true end-to-end SLI, catches silent failures that per-component metrics miss entirely, and gives you something concrete to describe when asked "how do you know your platform is healthy?". Write it in Python in about eighty lines.

tools/canary.py – the end-to-end probe (core logic)

```
import time, uuid, os
import clickhouse_connect
from opentelemetry import trace, metrics
from opentelemetry.sdk.trace import TracerProvider

tracer = trace.get_tracer("beacon.canary")
meter = metrics.get_meter("beacon.canary")
roundtrip = meter.create_histogram("beacon.canary.roundtrip", unit="s")
outcome = meter.create_counter("beacon.canary.probes")

ch = clickhouse_connect.get_client(host="clickhouse", database="otel")

def probe(timeout_s: float = 120.0) -> None:
    marker = str(uuid.uuid4())
    emitted = time.time()

    with tracer.start_as_current_span("beacon.canary.probe") as span:
        span.set_attribute("beacon.canary.marker", marker)

    # Poll the store until the span is queryable. This measures what the user feels:
    # not "did the collector accept it" but "can I find it".
    while time.time() - emitted < timeout_s:
        rows = ch.query(
            "SELECT count() FROM otel_traces "
            "WHERE SpanAttributes['beacon.canary.marker'] = %(m)s "
            "AND Timestamp > now() - INTERVAL 10 MINUTE",
            parameters={"m": marker}).result_rows
        if rows and rows[0][0] > 0:
            roundtrip.record(time.time() - emitted)
            outcome.add(1, {"result": "success"})
            return
        time.sleep(2)

    # A failed probe is a valid event for the SLI denominator – do not silently skip it.
    outcome.add(1, {"result": "timeout"})

if __name__ == "__main__":
    while True:
```

```
probe()  
time.sleep(30)
```

9. Platform SLIs, SLOs and error budgets

Now define SLOs for Beacon — for the platform, not for the tenants. This is the section to rehearse out loud.

The four SLOs every telemetry platform needs

SLO	SLI definition (good ÷ valid)	Target	Why this one
Ingest availability	OTLP export requests answered non-5xx and not rate-limited ÷ all OTLP export requests	99.9% 28d	If you cannot accept data, nothing else about the platform matters. Measured at the gateway's receiver.
Ingest freshness	Canary probes queryable within 60s ÷ all canary probes	99% 28d	Accepting data you never make queryable is silent failure. This is the SLI that catches it.
Query latency	Dashboard-representative queries completing < 5s ÷ all such queries	99% 28d	The read path is what users experience during an incident, when it is also under most load.
Alert delivery	Alert notifications delivered to the notification channel within 60s of rule firing ÷ all firings	99.95% 28d	The tightest target, because a late alert is indistinguishable from no alert, and this path is what the incident process depends on.

Say this in the interview

"I deliberately do not set an SLO on data completeness, because sampling and filtering mean some loss is *intended*. Instead I make the intended loss explicit — published per tenant, visible on a dashboard — and I SLO the things where loss is never intended: acceptance, freshness, queryability and alert delivery. Conflating deliberate reduction with failure is how telemetry platforms end up with SLOs nobody trusts."

Burn-rate alerting, and why thresholds are the wrong tool

The naive approach alerts when the SLI dips below the SLO for five minutes. It pages you at 3am for a blip that consumed 0.4% of the month's budget, and it stays silent through a slow degradation that eats the entire budget over nine days. **Burn rate** fixes both: it measures how fast you are consuming the error budget relative to the rate that would exactly exhaust it over the window.

Burn rate

Burn rate = observed error ratio ÷ (1 – SLO). A burn rate of 1 exhausts the budget precisely at the end of the window. A burn rate of 14.4 exhausts it in 2 days; a rate of 6 in about 4.7 days. You alert on *combinations* of a fast-burn short window and a slow-burn long window, each paired with a shorter confirmation window to reduce reset time and false positives.

Severity	Long window	Short window	Burn rate	Budget consumed before firing	Action
Page	1 hour	5 minutes	14.4	2%	Wake someone up
Page	6 hours	30 minutes	6	5%	Wake someone up
Ticket	1 day	2 hours	3	10%	Next business day
Ticket	3 days	6 hours	1	10%	Backlog, review at the weekly

platform/prometheus/rules/slo-ingest.yaml

```
groups:
- name: beacon-ingest-availability-slo
  rules:
    # Recording rules first: compute the ratio once, use it in four alerts.
    # This is also how you keep dashboards fast and definitions consistent.
    - record: beacon:ingest_error_ratio:rate5m
      expr: |
        sum(rate(otelcol_receiver_refused_spans_total{job="otel-gateway"}[5m]))
        /
        clamp_min(sum(rate(otelcol_receiver_accepted_spans_total{job="otel-gateway"}[5m]))
          + sum(rate(otelcol_receiver_refused_spans_total{job="otel-gateway"}[5m])), 1)

    - record: beacon:ingest_error_ratio:rate1h
      expr: |
        sum(rate(otelcol_receiver_refused_spans_total{job="otel-gateway"}[1h]))
        /
        clamp_min(sum(rate(otelcol_receiver_accepted_spans_total{job="otel-gateway"}[1h]))
          + sum(rate(otelcol_receiver_refused_spans_total{job="otel-gateway"}[1h])), 1)

    # Fast burn: 14.4x over 1h, confirmed by 5m. Fires after ~2% of a 28d budget.
    - alert: BeaconIngestErrorBudgetFastBurn
      expr: |
        beacon:ingest_error_ratio:rate1h > (14.4 * 0.001)
        and
        beacon:ingest_error_ratio:rate5m > (14.4 * 0.001)
      for: 2m
      labels:
        severity: page
        slo: ingest-availability
      annotations:
        summary: "Ingest burning error budget 14.4x – 2% consumed"
        # Every page carries its runbook. No exceptions. This is an auditable standard.
        runbook_url: "https://git.example/beacon/docs/runbooks/ingest-backlog.md"
        dashboard_url: "https://grafana.example/d/beacon-ingest"
        impact: "Tenants are being rate-limited; spans are being lost at source."

    # Slow burn: 3x over 1 day. A ticket, not a page. Symptom of gradual growth.
    - alert: BeaconIngestErrorBudgetSlowBurn
      expr: |
        beacon:ingest_error_ratio:rate1d > (3 * 0.001)
        and
        beacon:ingest_error_ratio:rate2h > (3 * 0.001)
      for: 15m
      labels: { severity: ticket, slo: ingest-availability }
```

The error budget policy — the part candidates forget

A number with no consequence is theatre. Write the policy down and bring it: **>75% budget remaining** — ship freely, take risks, run game days. **25–75%** — normal change control; reliability work prioritised alongside features. **<25%** — reliability items take priority in the next sprint; non-essential platform changes deferred. **Exhausted** — change freeze on the affected component except reliability fixes and security patches, until the budget recovers, with a named owner and a written recovery plan. Then add the escape hatch, because rigid policies get ignored: an explicitly documented override, signed off by a named accountable person, recorded in the same repo.

10. ClickHouse for telemetry

ClickHouse is named in the advert, so expect at least one question. You do not need to be a ClickHouse expert; you need to understand *why a columnar store is the right shape for telemetry* and what goes wrong operationally.

Why columnar for observability

Telemetry queries touch a few columns across enormous numbers of rows ("p99 duration for service X over 6 hours"). A column store reads only the columns referenced, compresses each column with a codec suited to its data type, and skips whole granules using sparse primary-key indexes. Compression ratios of 10–30× on span data are ordinary. The trade: point lookups and frequent small writes are comparatively poor — which is precisely why the collector's `batch` processor is not optional.

platform/clickhouse/ddl/01-traces.sql

```
CREATE TABLE otel.otel_traces
(
    Timestamp          DateTime64(9) CODEC(Delta, ZSTD(1)),
    TraceId            String        CODEC(ZSTD(1)),
    SpanId             String        CODEC(ZSTD(1)),
    ParentSpanId       String        CODEC(ZSTD(1)),
    SpanName           LowCardinality(String) CODEC(ZSTD(1)),
    ServiceName        LowCardinality(String) CODEC(ZSTD(1)),
    Duration            UInt64        CODEC(ZSTD(1)),
    StatusCode         LowCardinality(String),
    ResourceAttributes Map(LowCardinality(String), String) CODEC(ZSTD(1)),
    SpanAttributes     Map(LowCardinality(String), String) CODEC(ZSTD(1)),

    -- Skip indexes make attribute lookups tolerable without exploding the schema
    INDEX idx_trace_id  TraceId          TYPE bloom_filter(0.01) GRANULARITY 1,
    INDEX idx_span_attr mapKeys(SpanAttributes) TYPE bloom_filter(0.01) GRANULARITY 1
)
ENGINE = MergeTree
-- Partition by day: makes TTL drops a metadata operation instead of a rewrite.
-- Partition by hour and you will drown in parts; by month and you cannot drop granularly.
PARTITION BY toDate(Timestamp)
-- Order by what you filter on, cheapest-cardinality first. This ordering IS your index.
ORDER BY (ServiceName, SpanName, toUnixTimestamp(Timestamp))
-- Tiered retention: hot 7 days on fast disk, warm 30 on object storage, then gone.
TTL toDate(Timestamp) + INTERVAL 7 DAY TO VOLUME 'warm',
```

```
toDateTimestamp(Timestamp) + INTERVAL 30 DAY DELETE
SETTINGS index_granularity = 8192, ttl_only_drop_parts = 1;
```

Queries you should be able to write from memory

```
-- p50/p95/p99 latency by service over 6 hours – the RED "duration"
SELECT ServiceName,
       quantile(0.50)(Duration/1e6) AS p50_ms,
       quantile(0.95)(Duration/1e6) AS p95_ms,
       quantile(0.99)(Duration/1e6) AS p99_ms,
       count() AS spans
FROM otel.otel_traces
WHERE Timestamp > now() - INTERVAL 6 HOUR
GROUP BY ServiceName ORDER BY p99_ms DESC;

-- Which tenant is costing you money? Ingest volume and storage by tenant.
-- This query is the foundation of showback and of every noisy-neighbour conversation.
SELECT ResourceAttributes['beacon.tenant'] AS tenant,
       count() AS spans,
       formatReadableSize(sum(length(toJSONString(SpanAttributes)))) AS approx_attr_bytes
FROM otel.otel_traces
WHERE Timestamp > now() - INTERVAL 1 DAY
GROUP BY tenant ORDER BY spans DESC LIMIT 20;

-- Cardinality watch: attribute keys with the most distinct values.
-- Run this daily; it is your early warning for a metrics-store outage.
SELECT key, uniqCombined(value) AS distinct_values
FROM otel.otel_traces
ARRAY JOIN mapKeys(SpanAttributes) AS key, mapValues(SpanAttributes) AS value
WHERE Timestamp > now() - INTERVAL 1 HOUR
GROUP BY key ORDER BY distinct_values DESC LIMIT 25;

-- Operational health of the store itself: too many parts = insert pattern problem
SELECT table, partition, count() AS parts, formatReadableSize(sum(bytes_on_disk))
FROM system.parts WHERE active GROUP BY table, partition
HAVING parts > 100 ORDER BY parts DESC;
```

The ClickHouse failure mode to be able to describe

Too many parts. ClickHouse writes each insert as an immutable part and merges them in the background. Insert too frequently in small batches and part creation outruns merging; the merge queue grows, queries slow because they must read hundreds of parts, and eventually inserts are rejected outright. The fix is upstream — larger batches at the collector, fewer concurrent writers, appropriate partitioning — which is a lovely illustration of the platform SRE point that **the cause of a storage incident usually lives in the pipeline configuration.**

11. Alert quality and event correlation

"Alert quality measures" appears in the desirable list. Most engineers have never measured alert quality, so having a framework here is a genuine differentiator.

What makes an alert good

1

It is about a symptom, not a cause

Alert on "ingest freshness exceeded 60s", not "CPU on gateway-3 is high". High CPU may be harmless; stale telemetry never is. Cause-based alerts multiply as your system grows; symptom-based alerts stay constant.

2

It is actionable

If the recipient's only possible response is "acknowledge and wait", it should not have paged. Test: can you name the first three actions? If not, it is a dashboard, not an alert.

3

It carries context

Runbook link, dashboard link, plain-English impact statement, affected tenants, and the query that produced it. Enforce this in CI — an alert rule without `runbook_url` fails the build.

4

It is tied to user impact

Ideally derived from an SLO burn rate. If you cannot connect an alert to an SLO, ask what it is protecting; often the honest answer is "nothing, it was added after an incident in 2021".

5

It is rare

A rota that receives more than roughly two pages per shift stops reading them. Alert fatigue is not a personal failing, it is a design defect, and it is measurable.

Measuring alert quality

Measure	Definition	Target / signal
Precision	Alerts that led to a genuine action ÷ all alerts fired	> 80%. Below 50% and the rota has stopped trusting the system.
Recall (detection coverage)	Incidents detected by an alert ÷ all incidents	> 90%. Incidents first reported by a user are a coverage gap; track them separately as a headline number.
Self-resolving rate	Alerts that cleared without human action ÷ all alerts	< 20%. High values mean thresholds are too tight or <code>for:</code> durations too short.
Alert-to-incident ratio	Alerts fired ÷ incidents declared	Track the trend, not the value. A rising ratio means noise is growing.
Pages per shift	Out-of-hours pages ÷ shifts	< 2. Report per rota; it is a wellbeing metric as much as a technical one.
Runbook coverage	Paging alerts with a valid, tested runbook link ÷ all paging alerts	100%, enforced in CI, verified by link-checking in the pipeline.
Time to first useful signal	Incident start → first alert that pointed at the right component	Falling trend. This is your observability product's true KPI.

Event correlation and deduplication (the BigPanda-shaped problem)

When a single underlying failure produces four hundred alerts across dozens of services, a correlation layer groups related events into one actionable incident using shared attributes (time proximity, topology, shared dependency, common labels) and suppresses duplicates. Understand the concepts and you can talk about any tool in this space: **deduplication** (identical events collapse to one with a count), **grouping/clustering** (related-but-distinct events form one incident), **enrichment** (attaching CMDB, ownership and topology data at event time), **suppression** (maintenance windows, known-issue silences), and **flapping detection**. Alertmanager gives you dedup, grouping and inhibition natively — build with it, then explain how a commercial correlation engine extends the same ideas with topology and machine learning.

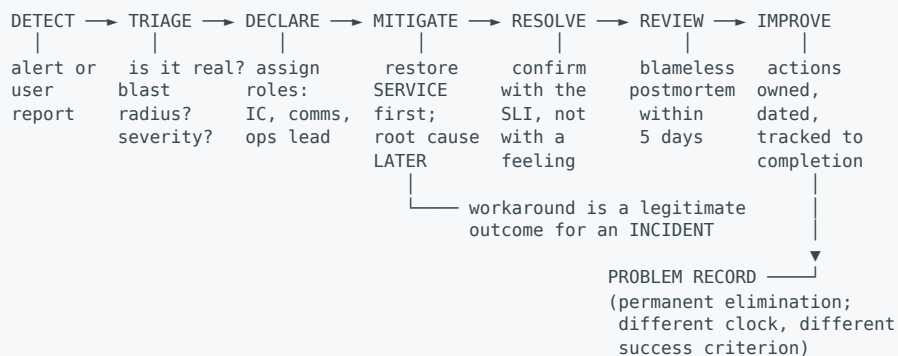
platform/alertmanager/config.yaml — dedup, grouping and inhibition

```
route:
  # Group by the dimensions that make one human action. Getting this wrong is
  # the number one cause of alert storms reaching a phone.
  group_by: ['alertname', 'cluster', 'slo']
  group_wait: 30s      # wait for siblings before the first notification
  group_interval: 5m    # then batch further changes to the group
  repeat_interval: 4h   # re-notify an unresolved group this often
  receiver: platform-oncall
  routes:
    - matchers: ['severity="ticket"']
      receiver: platform-backlog
      group_interval: 1h

# Inhibition: if the whole gateway is down, do not also page about every tenant's
# freshness SLO. Cause suppresses consequence. This is correlation in miniature.
inhibit_rules:
  - source_matchers: ['alertname="BeaconGatewayDown"']
    target_matchers: ['slo="ingest-freshness"']
    equal: ['cluster']
```

12. Incident and problem management

The advert asks for incident response, triage, problem management and root cause analysis, and then — crucially — "drive follow-up actions that reduce repeat issues". That last clause is about the loop, not the heroics.



Severity model — bring one to the interview

Sev	Definition for a telemetry platform	Response	Example
Sev1	Total loss of ingest or alert delivery across all tenants; the organisation is blind	Immediate page, IC assigned, comms bridge, executive notification	Gateway fleet OOM-looping; no telemetry has landed for 12 minutes
Sev2	Major degradation or complete loss for a critical tenant; SLO burning at page rate	Page, IC assigned, status update every 30 minutes	ClickHouse rejecting inserts; queue draining to persistent storage with 40 minutes of headroom
Sev3	Partial degradation with a workaround; single non-critical tenant affected	Business hours, ticket, no bridge	One tenant's dashboards slow after a bad query; others unaffected
Sev4	Minor or cosmetic; no SLO impact	Backlog	A dashboard panel legend renders wrongly after an upgrade

Incident roles, stated crisply

Incident Commander — owns the incident, makes decisions, does not debug. The most common failure in small teams is the IC getting sucked into a terminal and the coordination collapsing.

Communications lead — owns stakeholder updates on a fixed cadence, so the IC is not interrupted. **Operations lead** — the person with their hands on the system, executing changes the IC has approved. **Scribe** — maintains the timeline in real time; this is what makes the postmortem possible and it is impossible to reconstruct afterwards. In a small team one person may hold several roles, but naming them explicitly at declaration time is what stops a three-way conversation becoming a nine-way one.

The postmortem template you will actually use

docs/postmortems/2026-04-18-ingest-backlog.md

```
# Ingest backlog and 22 minutes of telemetry loss — 18 April 2026
```

```
| Field | Value |
|---|---|
| Severity | Sev2 |
| Duration | 14:02–14:47 UTC (45m); data loss 14:25–14:47 (22m) |
| Detected by | BeaconIngestErrorBudgetFastBurn (alert), 14:04 |
| Time to detect | 2 minutes |
| Time to mitigate | 45 minutes |
| Error budget consumed | 31% of the 28-day ingest-availability budget |
| Tenants affected | 41 of 46 |
```

Impact

State it in the user's language, not yours. "Between 14:25 and 14:47 no traces or logs were queryable for 41 tenants. Two teams were mid-incident and lost their diagnostic view; one escalated to a Sev1 on their own service as a result."

Timeline

All times UTC. Include what people BELIEVED at each point, not just what was true — that is where the diagnosis-time improvements come from.

- 13:58 Tenant `pricing-svc` deploys v2.3 which adds `sku` as a metric label.
- 14:02 Gateway memory climbs; memory_limiter begins refusing.
- 14:04 Fast-burn alert fires. On-call acknowledges.
- 14:11 On-call believes ClickHouse is the cause (recent upgrade) and investigates the store for 9 minutes. ****This was the diagnosis dead end.****
- 14:20 Cardinality dashboard checked; `sku` label identified.

- 14:25 Sending queue reaches capacity; drops begin.
- 14:31 Tenant asked to roll back; they need a change approval.
- 14:44 Emergency gateway-side filter deployed dropping the offending label.
- 14:47 Ingest recovers; queue drains over the following 6 minutes.

Root cause

Use "contributing factors", plural. Single-root-cause thinking is the main reason incidents repeat.

1. No per-tenant cardinality admission control at the gateway.
2. The cardinality dashboard was not linked from the ingest runbook, costing 9 minutes.
3. Gateway memory headroom was sized for mean load, not for a single-tenant spike.
4. Tenant had no pre-merge check for new metric labels.

What went well

Say this genuinely. The fast-burn alert worked exactly as designed; persistent queueing prevented loss for the first 23 minutes.

Actions

#	Action	Type	Owner	Due	Ticket
1	Per-tenant series-cardinality limiter at gateway	Prevent	A. Rivera	02 May	BEA-812
2	Link cardinality dashboard from ingest runbook	Detect	A. Rivera	21 Apr	BEA-813
3	Add label-cardinality check to tenant CI template	Prevent	J. Okafor	09 May	BEA-814
4	Pre-approved emergency filter change procedure	Mitigate	J. Okafor	25 Apr	BEA-815

Classify every action as Prevent / Detect / Mitigate. If all your actions are "Prevent", you have not thought about the next unknown failure.

Write one of these for real

Cause this exact incident on Beacon. Add a high-cardinality label to your Python service, watch the gateway refuse, observe the queue fill, mitigate with a gateway filter, then write the postmortem above with your genuine timeline. Bring it to the interview as a document. Very few candidates will have one, and it evidences at least five of the advert's requirements simultaneously.

13. Runbooks that work at 3am

"Experience writing or maintaining operational documentation such as runbooks" is a stated requirement. The bar is not "we have a wiki". The bar is: can a competent engineer who has never seen this system restore service from your document at 3am?

Property	Why it matters
Starts with impact and severity	The reader decides in ten seconds whether to escalate before reading further
Diagnosis is a decision tree, not prose	At 3am nobody reads paragraphs. "If X, go to section 3" works.
Commands are copy-pasteable	Placeholders in <code><angle brackets></code> , every command complete, no "then configure appropriately"
States what is safe and what is not	Explicitly mark destructive steps and the approval needed. In a regulated environment, name the change record.
Has an explicit escalation point	"If not resolved in 20 minutes, escalate to X" — permission to stop trying alone
Lives in Git next to the code	Reviewed in PRs, versioned with the system, and provably current
Has a last-tested date	An untested runbook is a hypothesis. Test them in game days and record the date.

```
# Runbook: ingest backlog / telemetry not appearing

**Impact**: tenants cannot see recent telemetry. If sustained >30m, data is permanently lost.
**Severity**: Sev2, or Sev1 if all tenants affected or alert delivery is impaired.
**Last tested**: 2026-04-02 (game day GD-14) **Owner**: platform-sre

## 1. Confirm and size the problem (2 minutes)
```
kubectll -n beacon get pods -l app=otel-gateway
curl -s http://gateway:8888/metrics | grep -E 'queue_size|queue_capacity|refused'
```

Ingest dashboard: https://grafana.example/d/beacon-ingest
- Queue near capacity → go to §2 (downstream problem)
- Refused spans rising, queue normal → go to §3 (admission/memory problem)
- Both normal but no data queryable → go to §4 (store or query path)

## 2. Exporter cannot drain (downstream)
Check the store first – the collector is usually the victim, not the culprit.
```
clickhouse-client -q "SELECT count() FROM system.parts WHERE active AND table='otel_traces'"
clickhouse-client -q "SELECT * FROM system.merges FORMAT Vertical"
df -h /var/lib/clickhouse
```

- Disk >85% → §2a expedite TTL - Parts >300 → §2b batching problem - Merges stuck → escalate

## 3. Gateway refusing at admission
Find the tenant responsible before you scale – scaling to absorb an abusive tenant
just moves the failure an hour later and costs money.
```
clickhouse-client -q "SELECT ResourceAttributes['beacon.tenant'] t, count() c
FROM otel.otel_traces WHERE Timestamp > now() - INTERVAL 15 MINUTE
GROUP BY t ORDER BY c DESC LIMIT 10"
```

**SAFE**: scale gateway replicas (`kubectll scale --replicas=N`), no approval needed.
**REQUIRES APPROVAL**: emergency filter dropping a tenant's data – raise an emergency
change record, notify the tenant in #beacon-status BEFORE applying, record in the incident.

## 5. Escalation
20 minutes without mitigation, or any suspected data loss → escalate to the platform
lead and declare Sev1 if all tenants are affected.
```

14. Level 2 cheatsheet

Query / command	Use
<code>rate(otelcol_receiver_refused_spans_total[5m])</code>	Are we shedding at the door?
<code>otelcol_exporter_queue_size / otelcol_exporter_queue_capacity</code>	Headroom before drops — your best leading indicator
<code>topk(10, count by (__name__)) ({__name__=~".+"}))</code>	Which metric names dominate your series count
<code>count(count by (label) (metric))</code>	Cardinality of a single label — run before approving a new one
<code>histogram_quantile(0.99, sum by (le) (rate(bucket[5m])))</code>	p99 from a histogram; note the <code>sum by (le)</code> or it is wrong

Query / command	Use
<code>predict_linear(node_filesystem_avail_bytes[6h], 4*3600) < 0</code>	Disk will fill within 4 hours — predictive, not reactive
<code>amtool alert query --alertmanager.url=...</code>	What is currently firing, from the CLI
<code>amtool silence add alertname=X --duration=2h --comment="..."</code>	Silence with an audit trail — never silence without a comment
<code>SELECT * FROM system.query_log WHERE query_duration_ms > 5000</code>	ClickHouse slow queries — the read-path SLI's source data
<code>SELECT * FROM system.asynchronous_metrics WHERE metric LIKE '%Memory%'</code>	Store memory pressure

Level 2 references

-  [SRE Workbook — Alerting on SLOs](#) — the definitive treatment of multi-window multi-burn-rate alerting. Read it twice.
-  [SRE Workbook — Implementing SLOs](#)
-  [Collector configuration reference](#) and [Collector internal telemetry](#) — the second link is your meta-monitoring source of truth.
-  [Collector deployment patterns](#) — agent, gateway, and when each is right.
-  [ClickHouse documentation](#) — read MergeTree, TTL, and the "too many parts" material specifically.
-  [PagerDuty Incident Response documentation](#) — free, practical, and the source of most organisations' role definitions.
-  [PagerDuty Postmortem guide](#)
-  [danluu/post-mortems](#) — a curated list of real public postmortems. Read ten. Nothing teaches failure modes faster.
-  [Prometheus alerting best practices](#)
-  [How They SRE](#) — a curated collection of public SRE practices from large engineering organisations.
-  [Sloth and OpenSLO](#) — generate Prometheus burn-rate rules from a declarative SLO spec. This is SLO-as-code, and it is exactly the sort of self-service capability the advert wants.

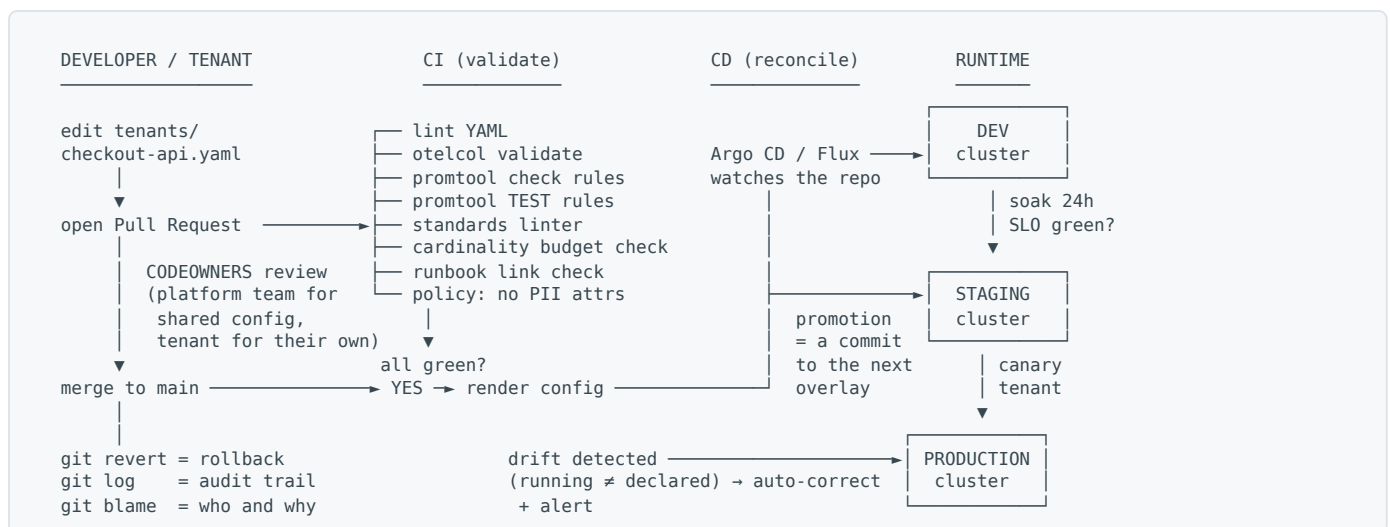


Level 3 • Platform Engineering at Scale

GitOps, self-service, telemetry economics, multi-tenancy, readiness and the realities of a regulated financial-services estate. Target: three weeks. This level is where you stop being an operator and start being a platform owner.

15. GitOps and environment promotion

The advert asks you to "improve automation and GitOps-based workflows to reduce manual work". In practice that means: nobody, including you, changes production by hand.



The promotion model, and how to talk about it

Environment promotion should move an *artefact*, not re-render one. Same rendered configuration, different values, promoted by a commit that changes a version reference. Three anti-patterns to name in the interview because naming them proves you have done it:

- **Re-templating per environment.** If production renders from source at deploy time, you never actually tested what production runs. Render once, promote the render.
- **Environment branches.** `main` / `staging` / `prod` branches drift, produce merge conflicts that get resolved wrongly under time pressure, and destroy the audit story. Use directories or overlays on one branch.
- **Secrets in the repository.** Sealed Secrets, External Secrets Operator, AWS Secrets Manager or Azure Key Vault with a CSI driver. In a financial-services context, be ready to talk about rotation and separation of duties too.

`.github/workflows/ci.yaml` – the gates that make self-service safe

```

name: beacon-ci
on: { pull_request: { paths: ['platform/**', 'tenants/**'] } }

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # 1. The collector config must be structurally valid. Catching this in CI
      # instead of at rollout is the difference between a PR comment and an incident.
      - name: Validate collector configuration
        run: |
          docker run --rm -v "$PWD/platform:/cfg" \
            otel/opentelemetry-collector-contrib:0.146.0 \
            validate --config=/cfg/collector-gateway.yaml

      # 2. Alert rules must parse AND their logic must be unit-tested.
      - name: Check and test alerting rules
        run: |
          promtool check rules platform/prometheus/rules/*.yaml
          promtool test rules platform/prometheus/tests/*.yaml

      # 3. Telemetry standards: the linter that replaces you answering questions in chat.
      - name: Enforce telemetry standards
        run: python tools/validate-standards.py tenants/

      # 4. Every paging alert must have a runbook that resolves.
      - name: Runbook link coverage
        run: python tools/check-runbook-links.py

      # 5. Render the config the way production will consume it and diff it into the PR,
      # so the reviewer sees the EFFECT of the change, not just the source.
      - name: Render and post config diff
        run: python tools/generate-config.py --diff-against origin/main

```

tools/validate-standards.py – the linter that turns a standard into a control

```

"""Enforce Beacon telemetry standards at pull-request time.
A written standard nobody enforces is a wish. A linter is a control."""
import sys, re, pathlib, yaml

# Attributes that must never be emitted – PII and unbounded-cardinality keys.
FORBIDDEN_ATTRS = {
    "customer.email", "customer.name", "user.ssn", "card.pan",
    "http.request.header.authorization", "session.token",
}

# Attributes safe on spans but catastrophic as metric labels.
UNBOUNDED_AS_METRIC_LABEL = {"trace_id", "span_id", "user.id", "order.id", "sku", "session.id"}
ATTR_NAME = re.compile(r"^[a-z][a-z0-9_]*(\.[a-z][a-z0-9_]*)+$") # semconv style
REQUIRED_RESOURCE = {"service.name", "service.namespace",
    "deployment.environment.name", "beacon.tenant", "beacon.cost.centre"}

def check(path: pathlib.Path) -> list[str]:
    m = yaml.safe_load(path.read_text())
    errs: list[str] = []

    missing = REQUIRED_RESOURCE - set(m.get("resource_attributes", {}))
    if missing:
        errs.append(f"missing required resource attributes: {sorted(missing)}")

    for attr in m.get("declared_attributes", []):
        if attr in FORBIDDEN_ATTRS:
            errs.append(f"'{attr}' is forbidden (PII or secret material)")
        if not ATTR_NAME.match(attr):

```

```

        errs.append(f"'{attr}' does not follow semantic-convention naming")

    for metric in m.get("metrics", []):
        bad = set(metric.get("labels", [])) & UNBOUNDED_AS_METRIC_LABEL
        if bad:
            errs.append(f"metric '{metric['name']}' uses unbounded labels {sorted(bad)}; "
                        f"put these on spans instead")

    # Estimated cardinality budget: the product of declared label cardinalities.
    for metric in m.get("metrics", []):
        est = 1
        for lbl, card in metric.get("estimated_label_cardinality", {}).items():
            est *= card
        if est > m.get("cardinality_budget", 10000):
            errs.append(f"metric '{metric['name']}' estimated {est} series, "
                        f"over tenant budget {m.get('cardinality_budget', 10000)}")

    return errs

if __name__ == "__main__":
    failed = False
    for f in pathlib.Path(sys.argv[1]).glob("*.yaml"):
        if f.name.startswith("_"):
            continue
        for e in check(f):
            print(f"::error file={f}::{e}")          # renders inline on the PR
            failed = True
    sys.exit(1 if failed else 0)

```

Why this script wins interviews

It demonstrates, in one artefact: enforcing shared standards, self-service enablement, cardinality control, PII handling in a regulated context, automation replacing manual review, and infrastructure-as-code discipline. That is six advert bullets in ninety lines of Python. Bring it up unprompted when they ask how you help teams adopt standards.

16. Self-service and golden paths

The measure of a platform team is what happens when a new team wants to onboard. If the answer involves a meeting with you, the platform does not scale. Target: **a team onboards by raising one pull request and nobody on the platform team touches it.**

tenants/_template.yaml – the golden path, in one file

```

# Copy this file to tenants/<your-service>.yaml, fill it in, raise a PR.
# CI validates it. Merge provisions everything. Nobody has to help you.
# Questions: #beacon-support (we answer in the docs, then link you to them).

tenant: retail-checkout
owner:
  team: retail-platform
  slack: "#retail-platform"
  escalation rota: pagerduty://retail-platform-primary # must resolve, checked in CI
  cost_centre: CC-4471

services:
  - { name: checkout-api, language: dotnet, criticality: tier-1 }
  - { name: checkout-worker, language: dotnet, criticality: tier-2 }

# Criticality drives everything downstream: retention, sampling, quota, alert routing.

```

```

# One declared field, many derived behaviours – that is what makes a golden path cheap.
telemetry:
  signals: [traces, metrics, logs]
  sampling:
    strategy: tail          # tail | head | none – tier-1 defaults to tail
    baseline_percentage: 5
    always_keep: [errors, slow]
  retention:
    traces: 14d             # max 30d without an approved exception
    logs: 30d
    metrics: 400d          # regulatory minimum for tier-1 in this estate
  cardinality_budget: 25000

quotas:
  spans_per_second: 5000
  log_bytes_per_second: 2000000
  on_exceed: throttle      # throttle | drop-lowest-priority | page-tenant

slos:
  # The tenant declares their own SLOs. The platform GENERATES the recording rules,
  # burn-rate alerts and an SLO dashboard from this. The platform team never writes
  # a tenant SLO – we provide the machinery, they provide the intent.
  - name: checkout-availability
    sli: 'sum(rate(http_server_request_duration_seconds_count{service="checkout-api",http_response_status_code!~"5.."}
[5m])) / sum(rate(http_server_request_duration_seconds_count{service="checkout-api"}{5m}))'
    objective: 99.9
    window: 28d

dashboards:
  starter_pack: [red-overview, slo-burndown, dependency-map]  # generated, editable copies

alert_routing:
  page: pagerduty://retail-platform-primary
  ticket: jira://RETAIL

```

The generated-not-authored principle

Notice what the platform team owns here: the schema, the generator, the validation, the defaults and the guard rails. What the tenant owns: their intent. This is the single most important structural idea in the whole role, and it maps directly to "ownership of dashboards, alerts, or telemetry for every application team is not what we are looking for here". You own the *machine that produces* dashboards and alerts. They own what goes in it.

Adoption is a metric, not a hope

Adoption metric	What it tells you
Tenants onboarded without platform-team involvement ÷ all onboardings	Whether the golden path actually works, or whether you are the golden path
Median time from first PR to first telemetry queryable	The real onboarding experience. Measure it; it is always worse than you think.
Support requests per tenant per month, tagged by cause	Your documentation backlog, prioritised by actual pain
Standards conformance score per tenant	Gives teams something to improve and you something to report
Percentage of services on the current template version	Your ability to change anything centrally ever again

Adoption metric	What it tells you
Proportion of tenant alerts using generated rules vs bespoke	Bespoke is fine; a rising proportion means your templates do not fit reality

17. Telemetry economics

"Data routing, data filtering, retention or cost management" is in the desirable list because telemetry cost is one of the top line items in a modern enterprise engineering budget, and it grows super-linearly with headcount unless someone owns it. That someone is this role.

The reduction hierarchy — apply in this order

- 1. Do not emit it.** Cheapest byte is the one never created. Health-check spans, debug logs in production, duplicated instrumentation from two agents on the same process.
- 2. Drop at the edge.** Filter in the agent, before network egress. This also saves cross-AZ transfer, which is often the hidden half of the bill.
- 3. Aggregate.** Convert high-volume logs into metrics at the collector. A million "request completed" log lines become one counter with a handful of labels.
- 4. Sample.** Tail sampling keeps errors and slow traces at 100% and the boring majority at 1-5%. Typical reduction: 90-95% of span volume for a small loss in statistical fidelity.
- 5. Tier and expire.** Hot storage for days, warm object storage for weeks, then delete. Match retention to actual query patterns — measure how far back people really look before you argue about it.
- 6. Only then, negotiate the contract.** Vendors love customers who skip steps 1-5.

Lever	Typical reduction	Fidelity cost	Where it lives
Drop health-check and metrics-scrape spans	20-35% of spans	None. Nobody has ever debugged with these.	Agent filter processor
Drop DEBUG logs in production	40-60% of log volume	Low, if you can raise verbosity on demand	Gateway filter, ideally dynamic
Logs → metrics conversion	70%+ on the converted stream	Loses per-event detail; keep a sampled residue	Collector connector/transform
Tail sampling at 5% baseline	90-95% of remaining spans	Aggregate stats need adjusting for sample rate; errors and slow traces unaffected	Gateway (needs trace-affinity routing)
Attribute pruning	10-30% of bytes	Cannot query on what you pruned — publish the list	Transform processor
Tiered retention	50-70% of storage cost	Older data slower to query	Store TTL to volume
Per-tenant quotas	Caps the tail risk	A tenant can lose data — must be visible to them, never silent	Gateway admission control

The trap in every cost question

If you answer a cost-reduction question purely with reduction techniques, you have failed half of it. The senior answer names the risk: **every byte you drop is a question you cannot answer later, and you will not know which question until an incident.** So you (a) never sample errors, (b) publish exactly what is dropped so nobody is surprised at 3am, (c) provide a documented emergency path to raise fidelity for a service under investigation, and (d) review the drop rules after any incident where the missing data slowed diagnosis. Cost management without that governance is just deferred incident cost.

Showback: making cost visible to the people who generate it

You cannot control what tenants emit by asking nicely. You can by showing them a monthly figure attributed to their cost centre. Build the query, build the dashboard, send the report. Behaviour changes within one cycle — and this is a concrete, quantifiable achievement to describe in an interview: "I introduced per-tenant showback and total ingest fell 38% in two months without a single mandate."

18. Multi-tenancy and isolation

Isolation concern	Mechanism	Failure it prevents
Ingest rate	Per-tenant rate limit at the gateway, keyed on <code>beacon.tenant</code> , state in Redis	One team's log flood consumes the whole pipeline
Cardinality	Per-tenant series budget with rejection and a notification to the tenant	The classic metrics-store outage
Query resources	ClickHouse quotas and settings profiles per user; query complexity limits	One analyst's unbounded scan starves the dashboards during an incident
Storage	Retention by tier and tenant; separate tables or projections for the largest tenants	Cheap data crowding out expensive-to-lose data
Access	Row-level policies or per-tenant datasources; Grafana teams and folders mapped to identity groups	Cross-tenant data exposure — a reportable event in financial services
Blast radius of change	Canary tenant, then percentage rollout of gateway config	A collector config change breaking all tenants simultaneously

Fairness, stated properly

Rate limiting is easy; *fair* rate limiting is the interesting problem. Under global pressure, a naive limiter penalises whoever arrives last rather than whoever is misbehaving. Better: per-tenant token buckets sized by declared quota, with borrowing from unused global headroom, and priority classes so that tier-1 error traces survive when tier-3 debug logs do not. If you can describe that trade-off, you are talking like someone who has run a shared platform.

19. Resilience, readiness and performance validation

The advert names "resilience checks and performance validation" explicitly. Here is what to build and be able to discuss.

Production readiness checklist for a new platform component

Category	Gate
Ownership	Named owning team, on-call rota that actually receives pages, escalation path documented
Service levels	At least one SLI measured for 4 weeks, an SLO with a written rationale, an error budget policy
Observability of itself	Health endpoint, self-metrics scraped by the independent meta-monitoring stack, structured logs, dashboard published
Alerting	Symptom-based alerts tied to the SLO, each with a runbook link, routed and tested end to end
Runbooks	Written, linked, exercised in at least one game day, dated
Capacity	Load-tested to 2× projected peak; known saturation point and the metric that shows you approaching it
Failure behaviour	Documented behaviour under dependency failure — degrade, queue, shed, or fail — and it is the behaviour you actually chose
Change and rollback	Deployed by pipeline only; rollback tested, not assumed; canary path defined
Data	Retention set, PII controls verified, backup/restore tested if state is authoritative
Security	Secrets from a manager not a file, least-privilege identity, TLS in transit, dependency scanning in CI

Game days: the exercises to run on Beacon

Injected failure	How to inject it	What you must learn
Store unavailable	<code>docker compose stop clickhouse</code>	How long the persistent queue buys you; whether it drains cleanly; whether you alerted before users noticed
Store slow, not down	<code>tc qdisc add dev eth0 root netem delay 800ms</code>	Partial degradation is far harder than a clean outage — do your alerts distinguish them?
Cardinality explosion	Add <code>sku</code> as a metric label in pricing-svc	Detection time, whether admission control held, whether the runbook led you to the tenant quickly
Gateway OOM	Lower the memory limit under load	Whether <code>memory_limiter</code> sheds gracefully or the process dies; data loss during restart

Injected failure	How to inject it	What you must learn
Disk full	<code>fallocate -l 20G /var/lib/clickhouse/ballast</code>	Whether <code>predict_linear</code> warned you hours earlier; whether TTL expedition is documented
Poison payload	Send a malformed or 20MB OTLP request	Whether one bad tenant payload can wedge the pipeline for everyone
Config rollout gone wrong	Merge a valid-but-wrong filter that drops a tenant's data	Detection of <i>silent</i> loss, and how fast <code>git revert</code> actually restores service
Alert path failure	Block egress to the notification endpoint	Whether you find out. Most teams discover this during a real Sev1.

Run three of these for real and record them

For each: a hypothesis written before you start, the actual observed behaviour, time to detect, time to mitigate, and what you changed afterwards. A short document with three game days beats any amount of asserted experience, and it directly evidences "resilience checks", "performance validation", "root cause analysis" and "drive follow-up actions".

Performance validation of the pipeline

Load-test the platform, not just the tenant apps. Use the OpenTelemetry Collector's own load-generation tooling or a synthetic span generator, and establish these numbers for your own build so you can quote them:

- Spans per second per gateway replica at a given CPU allocation, before and after adding tail sampling
- The saturation point — where p99 receiver latency starts climbing and refusals begin
- Memory per 100k in-flight traces in the tail-sampling buffer (this is the number that determines your `num_traces` setting, and getting it wrong is the most common cause of gateway OOMs)
- Queue drain rate after a simulated 10-minute backend outage, which tells you your true recovery time
- ClickHouse insert throughput at various batch sizes — the data behind your `send_batch_size`

20. Working in a regulated / financial services environment

The advert lists this as desirable, which means at least one interviewer will probe whether you understand that "just push the fix" is not always available. Candidates from unregulated backgrounds often lose points here by sounding cavalier. You do not need to be a compliance expert — you need to show you can operate *fast* and *within* controls, and that you understand why the controls exist.

Constraint	What it means day to day	How a good platform SRE responds
Change management	Production changes need an approved record; some need a change advisory board	Pre-approve standard changes (scaling, rollback, documented runbook steps) so the rota is not blocked at 3am; keep a documented emergency change path; make GitOps history the evidence
Separation of duties	The person who writes a change may not be the person who approves it	CODEOWNERS and branch protection enforce it automatically, with the PR as the audit artefact
Auditability	You must show who changed what, when, and who approved it — often years later	Everything through Git; no console changes; alert-silence comments mandatory; access logged
Data residency and sovereignty	Telemetry may not leave a jurisdiction	Routing by <code>cloud.region</code> at the gateway; per-region stores; explicit exporter allow-lists
PII and payment data	Telemetry is a notorious leak path — logs and span attributes especially	Redaction at the gateway as a defence in depth, forbidden-attribute linting in CI, and a documented process for when it leaks anyway
Retention obligations	Some data must be kept for years; some must be deleted on schedule	Retention as a declared, reviewed property of each tenant, not an accident of a default setting
Operational resilience regimes	Regulators increasingly require evidence of tested recovery for important business services	Game days with written outcomes, tested runbooks with dates, documented impact tolerances — all of which you were doing anyway
Third-party and concentration risk	Vendor dependencies are scrutinised	OpenTelemetry as the instrumentation layer means backend migration is an exporter change, not a re-instrumentation programme — this is a genuinely strong argument to make

The line to have ready

"Controls slow you down when they are applied uniformly to everything. The work is to classify: pre-approve the low-risk, high-frequency changes with evidence attached automatically, so that the controls concentrate on the genuinely risky changes. GitOps helps enormously here, because the audit evidence is a by-product of how we work rather than a document someone writes afterwards."

21. Enablement and documentation

The final section of the advert's responsibilities is about making other teams self-sufficient. This is a real skill and it is assessed — often by asking you to explain something technical simply, or by asking how you would handle a team that keeps asking for help.

The four documents every platform needs

- **Onboarding guide** — from zero to first telemetry, tested by making a new joiner follow it without help
- **Standards** — what you must emit, what you must never emit, and *why*, because "why" is what makes standards stick
- **Runbooks** — for your own on-call rota
- **Reference** — the manifest schema, quota semantics, retention options, every knob explained once

Docs-as-code, non-negotiable

- Markdown in the same repository as the config it describes
- Reviewed in the same PR as the change — a change that alters behaviour without updating docs should not merge
- Link-checked and example-tested in CI
- Owned by CODEOWNERS, with a review date
- A wiki that drifts from reality is worse than no wiki: it costs trust as well as time

The support-request-to-documentation loop

Every question a tenant asks is a defect in your documentation, your defaults, or your error messages. Tag support requests by cause, count them monthly, and fix the top three at source rather than answering the same question forty times. Then measure the decline. Being able to say "requests about X fell from 22 a month to 3 after I rewrote the onboarding guide and improved the validation error message" is exactly the "practical approach to reducing manual work" the advert asks for, expressed as evidence rather than assertion.

Rehearse this out loud

"Explain what a trace is to a project manager." Ninety seconds, no jargon, one analogy, one reason they should care. Interviewers for platform roles ask a version of this constantly, because your entire job involves persuading other teams to change how they work. Practise it in the car.

Level 3 references

-  [OpenGitOps](#) — the four principles, stated canonically. Quote them accurately.
-  [Argo CD documentation](#) and [Flux documentation](#) — pick one and actually run it against your repo.
-  [Kubernetes concepts](#) — you need enough to be dangerous: workloads, services, probes, resources and limits, RBAC.
-  [kind](#) — a real cluster on your laptop in ninety seconds.
-  [Terraform documentation](#) — infrastructure-as-code for the cloud side; pair with [Bicep](#) if your estate is Azure-first.
-  [Grafana Alerting](#) and [Grafana provisioning](#) — provisioning is how dashboards become code.
-  [Cribl Stream documentation](#) — read the concepts even without a licence, so you can speak the vocabulary: sources, routes, pipelines, packs, destinations.
-  [Datadog documentation](#) — for the vendor-platform perspective on the same problems.
-  [Grafana Alloy](#) — an OTel-compatible collector distribution; useful comparison material.
-  [The Twelve-Factor App](#) — old, still the clearest statement of config-in-environment and disposability.

✿ Capstone • Project Beacon

Eight weeks, one repository, one running platform, and a portfolio of artefacts. Each week produces something you can show or discuss. If you have limited time, weeks 2, 4 and 6 carry the most interview weight.

Wk	Theme	Build	Artefact for the interview
1	Stack up	Compose stack: collector gateway, ClickHouse, Prometheus, Grafana, Redis. C# and Python tenant services emitting OTLP. Data visible end to end.	A repo with a README that explains what Beacon is, who it serves and how to onboard
2	Meta-monitoring	Independent Prometheus scraping collector self-metrics. The canary probe. A platform-health dashboard, provisioned as JSON from Git.	The platform-health dashboard and your answer to "how do you know the platform is healthy?"
3	SLOs	Four platform SLOs with recording rules and multi-window burn-rate alerts. A written SLO catalogue with rationale and an error budget policy.	docs/slo-catalogue.md — the document most candidates cannot produce
4	Break it	Three game days from the table in §19. Cause a real incident, respond to it, mitigate it.	A genuine postmortem with a timeline, contributing factors and Prevent/Detect/Mitigate actions
5	GitOps	kind cluster, Argo CD (or Flux), dev→staging overlays, the CI gates from §15, a deliberate drift-and-correct demonstration.	A PR that changes platform behaviour, showing the validation gates and the rendered diff
6	Self-service	Tenant manifest schema, generator producing collector routing plus SLO rules plus starter dashboards, standards linter, onboarding guide.	Onboard a third tenant purely by raising a PR, with no manual step. Demo this.
7	Economics	Tail sampling, filtering, logs-to-metrics, tiered retention. Measure ingest volume before and after. Per-tenant showback query and dashboard.	A one-page cost analysis with real numbers from your own platform
8	Polish and rehearse	Pin versions. Write the runbooks. Test the runbooks. Tidy the README. Then do the rehearsal in the next section.	A ten-minute walkthrough you can deliver from memory

The single highest-value thing in this document

Week 4. Almost every candidate can describe an incident they were *present* for. Very few arrive with a written postmortem for a system they built, broke and fixed themselves, including the eight minutes they wasted on a wrong hypothesis. That honesty about a dead end is what makes it credible, and it evidences incident response, problem management, root cause analysis and continuous improvement in one artefact.

How to present Beacon in ten minutes

1 Frame it as a product, not a lab (60s)

"Beacon is a multi-tenant telemetry platform. Its users are three tenant services owned by two notional teams. Its job is to accept, reduce, store and serve their telemetry, and to let them onboard without talking to me."

2 Show the platform-health dashboard first (90s)

Not the pretty tenant dashboard. Show queue depth, refusal rate, canary round-trip and burn rate. This immediately signals that you understand the difference this role is built around.

3 Walk one SLO end to end (2m)

Definition → SLI query → recording rule → burn-rate alert → runbook link → error budget policy. Complete chains are rare and memorable.

4 Tell the incident story (3m)

Timeline, the wrong hypothesis, the mitigation, the four contributing factors, the actions you completed. Then show the guard rail that now prevents it.

5 Demonstrate self-service (2m)

Open the tenant template, show the CI gates, show what merging generates. Say the sentence: "I own the machinery, the tenant owns the intent."

6 Close on what you would do differently at real scale (90s)

Trace-affinity load balancing for tail sampling across replicas, a real object-storage tier, per-tenant quota enforcement in a distributed limiter, multi-region routing for data residency. Knowing the limits of your own toy is a seniority signal.

Interview Rehearsal

Forty-five questions grouped by theme, a systems-design exercise with a worked skeleton, five live troubleshooting drills, a STAR story bank, and the failure modes that sink otherwise strong candidates.

22. Mapping the advert to your evidence

Before rehearsing answers, build this table for yourself. Every line of the advert should map to something you can point at. Gaps are where you should spend your remaining preparation time.

Advert requirement	Your evidence from Beacon	The sentence to say
Supporting production platforms in SRE/platform/infra/DevOps/ops	The whole capstone plus your actual work history	Lead with real work; use Beacon to fill gaps and show depth
Using observability data to investigate issues	Game days, the postmortem, the meta-monitoring dashboard	"I bisect the pipeline stage by stage; each stage has one metric that settles it."
Incident response, problem management, service readiness	Severity model, postmortem, readiness checklist	"Incident restores service; problem eliminates the cause. Different clocks."
Automation and infrastructure-as-code	CI gates, config generator, GitOps reconciliation	"Nobody changes production by hand, including me."
Cloud, container, Linux, networking, distributed systems	kind cluster, Compose, the Linux triage table, queueing behaviour	Answer with a strategy, then the commands
Communicating to engineering, ops and service stakeholders	Onboarding guide, standards doc, showback report	Have the ninety-second "what is a trace" explanation ready
Operational documentation	Runbooks with last-tested dates	"An untested runbook is a hypothesis."
Reducing manual work, helping teams use shared platforms	Self-service onboarding, the support-request-to-docs loop	"Every support question is a defect in docs, defaults or error messages."
OpenTelemetry / Grafana / ClickHouse / Cribl / Redis	All but Cribl built for real; Cribl understood conceptually	Never bluff. "I have not used it; here is the equivalent I have built."
SLOs, SLIs, error budgets, alert quality	SLO catalogue, burn-rate rules, alert-quality metrics	"An SLO without a policy is a decoration."
Telemetry pipelines, routing, filtering, retention, cost	The reduction hierarchy with measured before/after numbers	Quote your own percentages; specificity beats theory

Advert requirement	Your evidence from Beacon	The sentence to say
Regulated / financial services environment	Redaction, audit-by-Git, change classification	"Pre-approve the routine so controls concentrate on real risk."

23. Forty-five questions to rehearse

Platform reliability and observability

1. How is monitoring an observability platform different from monitoring an application?
2. Your platform is down. How do you know? Walk me through your meta-monitoring.
3. Users say "telemetry is missing". Where do you start?
4. Distinguish data that never arrived, data that was dropped deliberately, and data that arrived but is not queryable. How does your platform tell them apart?
5. What are the four golden signals, and which would you apply to a collector gateway?
6. When would you choose RED over USE?
7. Explain cardinality to someone who has never heard the term, then tell me how you would prevent an explosion.
8. What is the difference between head and tail sampling? What breaks when you scale tail sampling horizontally?
9. Why do processor order and batch placement matter in a collector pipeline?
10. How would you upgrade a fleet of collectors without losing telemetry?
11. A single tenant is generating 80% of your ingest volume. What do you do, in what order?
12. How do you detect data loss that produces no errors anywhere?
13. Why might a columnar store be right for traces and wrong for something else?
14. What is "too many parts" in ClickHouse and where does the real fix live?
15. How would you design telemetry ingest to survive a full outage of the storage backend?

SLOs, alerting and error budgets

16. Define SLI, SLO, SLA and error budget. Give a platform example of each.
17. What SLOs would you set for a telemetry platform, and why those?
18. Why not simply alert when the SLI drops below the SLO?
19. Explain multi-window multi-burn-rate alerting and what each window is for.
20. Your 99.9% SLO has 40 minutes of budget per 28 days. You have consumed 31% in one incident. What happens next?
21. A team wants a 99.99% SLO on your platform. How do you respond?
22. How do you measure whether your alerts are any good?
23. Your on-call rota is receiving twelve pages a night. What do you do first?
24. What makes an alert page-worthy rather than ticket-worthy?
25. How would you help three hundred teams define their own SLOs without writing them yourself?
26. What is an error budget policy and what do you do when someone wants to override it?

Incident response and problem management

- 27. Walk me through your first ten minutes on a Sev1.
- 28. What is the difference between an incident and a problem?
- 29. Who does what during an incident? What happens when the IC starts debugging?
- 30. Describe an incident where your first hypothesis was wrong. How did you notice?
- 31. What makes a postmortem blameless, and what does blameless *not* mean?
- 32. How do you make sure postmortem actions actually get done?
- 33. The same incident has happened three times. What is broken, beyond the technology?
- 34. How do you decide between restoring service quickly and preserving evidence for diagnosis?
- 35. Tell me about the worst incident you have been part of and what you changed afterwards.

GitOps, automation and platform engineering

- 36. What are the principles of GitOps, and which one do teams most often skip?
- 37. What is configuration drift and how do you detect it?
- 38. How do you promote a change from dev to production safely?
- 39. Why are environment branches a bad idea?
- 40. How do you make it safe for a tenant team to change platform configuration themselves?
- 41. How do you roll back a GitOps change that has already caused data loss?
- 42. How do you measure adoption of a platform capability?
- 43. A team refuses to adopt your standards. What do you do?
- 44. How do you cut telemetry cost by 40% without harming incident diagnosis?
- 45. How do you operate at speed inside a change-management regime?

Worked answer — question 3, because it is nearly guaranteed

"First I establish scope, because it changes everything: one tenant, one signal, or everyone? I check the platform-health dashboard before I touch anything, because if ingest refusals or queue depth moved *before* the report, the platform is the cause; if they moved after, I am probably looking at a consequence. Then I bisect the path. Is the agent receiving — accepted spans on the agent. Is the gateway accepting — accepted versus refused at the gateway. Is the exporter draining — queue size against capacity. Did it land — a direct count in ClickHouse for that service in the window. Is it queryable — my canary round-trip metric already answers that continuously. Each stage has exactly one metric that settles it, which is why I built the dashboard in that order. The most common answer, incidentally, is none of the above: the tenant deployed a change to their sampling configuration or renamed their service, so I also check whether `service.name` changed in the last hour before I go any deeper."

24. The systems design exercise

Likely prompt: "Design telemetry ingest for 4,000 services across three regions, two million spans per second at peak, in a regulated bank."

They are not testing whether you can build this. They are testing whether you ask the right questions, structure a large problem, and know where the hard parts are.

1 Clarify before designing (spend a real two minutes here)

Which signals? Retention requirements per signal — are any regulatory? Data residency constraints between regions? Who are the consumers: humans debugging, automated alerting, or analytics? What is the availability requirement for ingest versus query? Is there an existing vendor contract? What is the failure budget for data loss — is any loss acceptable?

2 Do the arithmetic out loud

2M spans/sec $\times$ ~400 bytes $\approx$ 800 MB/s raw, ~70 TB/day before reduction. At 10:1 compression and 5% tail sampling, roughly 350 GB/day retained. Now the design constraints are concrete, and the interviewer can see you size things rather than assert them.

3 Layer the architecture

Agent per node (enrich, batch, never lose data to a network blip) $\rightarrow$ regional gateway tier behind a load balancer $\rightarrow$ trace-affinity routing layer for tail sampling $\rightarrow$ regional store, with cross-region only for aggregates, never raw data, which also satisfies residency $\rightarrow$ query federation over regions $\rightarrow$ independent meta-monitoring per region plus one global view.

4 Name the hard parts before they ask

Trace affinity at this scale. Tail-sampling memory versus decision wait. Cardinality admission control that is fair rather than first-come. Query fan-out across regions when residency forbids moving raw data. Rolling collector upgrades without loss. Multi-tenant fairness under global saturation.

5 State the trade-offs you chose and what you gave up

"I chose regional isolation over a global store: it satisfies residency and limits blast radius, at the cost of harder cross-region trace queries and duplicated operational effort. If cross-region tracing turned out to be a genuine requirement rather than a nice-to-have, I would revisit it with a global index over regional data rather than moving the data."

6 Close with day two

How you would roll it out (canary tenant, then tier-3, then tier-1), how you would know it is healthy, what you would measure in the first month, and what you would expect to get wrong.

25. Live troubleshooting drills

Run each of these on Beacon until the diagnostic path is automatic. Time yourself. The interview version will be verbal, so narrate your reasoning aloud as you work.

Scenario	Set it up	What good looks like
"Traces stopped an hour ago for one team"	Add a filter that silently drops one <code>service.name</code>	You check whether other tenants are affected, spot that ingest metrics are healthy, and conclude it is policy not failure — then find the offending config in Git history
"Everything is slow"	<code>netem</code> delay on the ClickHouse container	You separate write-path from read-path symptoms, check queue depth and query duration, and identify partial degradation rather than assuming an outage
"The gateway keeps restarting"	Drop the memory limit and drive load	You check restart reason before logs, connect OOM to <code>num_traces</code> in tail sampling, and mitigate by reducing decision wait or scaling before changing sampling policy
"Dashboards are empty but ingest looks fine"	Change the timezone or the timestamp column mapping	You verify data is present with a direct query, then work backwards through the datasource to the panel — data exists, the question was wrong
"Alerts stopped firing"	Break the Alertmanager route	You immediately ask "how would we know?", then build the answer: a heartbeat alert that fires constantly and pages if it ever stops arriving

The dead-man's switch

The last drill has a specific expected answer that experienced platform engineers give and others do not: an always-firing heartbeat alert routed to an external watchdog, which pages when it stops receiving. If your alerting path fails silently, everything else you built is decoration. Have this ready.

26. STAR story bank

Prepare eight stories in Situation–Task–Action–Result form, each under two minutes, each with a number in the result. These cover the behavioural ground this advert implies.

Story	What it must contain
An incident you led or materially contributed to	Your specific actions, not the team's. A decision you made under uncertainty. What you changed afterwards.
A time your diagnosis was wrong	How you noticed, how quickly you abandoned the hypothesis, and what made you slow to see it. Self-awareness is the point.
Toil you automated	Hours saved per week, or manual steps eliminated. A number.
A repeat problem you eliminated permanently	Incident count before and after. This is the problem-management story the advert asks for directly.
Documentation that changed behaviour	Support requests before and after, or onboarding time reduced.
Persuading a team to adopt something	Resistance you met, what actually worked, and what you conceded. If you gave nothing up, it did not happen.
A trade-off between reliability and delivery speed	How you framed it for non-engineers, and the decision that resulted.
Something you broke	Own it plainly, describe the blast radius honestly, and explain the control that now prevents it. Interviewers trust candidates who have visibly learned something.

27. How candidates fail this interview

Failure mode	The fix
Answering as an application monitoring engineer	Reread §0. Every answer should be about the platform, its tenants and its standards.
Listing tools instead of describing problems	Lead with the problem, then the mechanism, then name the tool last. "I needed per-tenant admission control, so I..." beats "I've used Cribl."
Bluffing on a listed tool	Say you have not used it and pivot to the equivalent you have built. Interviewers probe; being caught costs the whole interview, and admitting it costs almost nothing.
Reciting SLO definitions without the trade-off	Always finish with the consequence: the policy, the budget, the conversation it forces.
Heroism as a narrative	"I was up until 4am fixing it" reads as a systems failure. Frame around detection, mitigation and prevention instead.
Blaming other teams	A noisy tenant is a missing guard rail. Say so — it is both truer and more impressive.
No numbers anywhere	Percentages, durations, volumes, counts. Even from your capstone. Specificity is the strongest credibility signal available.

Failure mode	The fix
Treating documentation as a chore	The advert mentions documentation and enablement four separate times. It is a core deliverable here, not an afterthought.
Dismissing governance	In financial services, "we'd just push it" ends the conversation. Show you can move fast <i>through</i> controls.
No questions at the end	See below.

Questions worth asking them

- How many teams are on the platform today, and what does onboarding currently cost your team in hours?
- What does the on-call rota actually look like — pages per week, and what are the top three recurring causes?
- Do you have SLOs on the platform itself today? What happens when a budget is exhausted?
- Where does the boundary sit in practice between this team and application teams? Where does it get argued about?
- What is the biggest source of toil for the team right now?
- How much of the estate is on OpenTelemetry versus legacy agents, and what is the migration story?
- What would you want the person in this role to have achieved by the end of their first six months?

28. Career roadmap and certifications

1

Weeks 1-2 • Fluency

Vocabulary cold. Beacon running. Traces flowing from C# and Python. Read SRE Book chapters 1-6 and the SRE Workbook chapters on implementing and alerting on SLOs.

2

Weeks 3-5 • Operating depth

Meta-monitoring, canary, four SLOs with burn-rate alerts, three game days, one real postmortem, runbooks written and tested.

3

Weeks 6-8 • Platform thinking

GitOps with Argo CD or Flux, CI gates, tenant self-service onboarding, cost reduction with measured numbers, documentation you would hand to a stranger.

4

Weeks 9+ • Rehearse and apply

The forty-five questions out loud. The ten-minute walkthrough from memory. The STAR bank. Then apply — and keep building, because the second interview will go deeper than the first.

Beyond the role

Senior platform SRE → platform lead or principal engineer. The branch points are observability architecture, reliability strategy across an organisation, or engineering management. All three are open from here.

Certification	Relevance	Why, honestly
Certified Kubernetes Administrator (CKA)	High	Hands-on and time-pressured, so it proves practical ability rather than recall. The best single certificate for this role.
Certified Grafana Associate	High	Grafana Labs' only official certification; directly relevant to a named tool in the advert.
AWS Certified DevOps Engineer – Professional	High	Covers monitoring, logging, IaC and incident response in the AWS idiom. Strong fit if the estate is AWS.
Microsoft AZ-400 DevOps Engineer Expert	High	The Azure equivalent; covers GitOps-adjacent practice, pipelines and observability with Azure Monitor.
Google Professional Cloud DevOps Engineer	Medium-high	The most SRE-native syllabus of the cloud certifications — SLOs, error budgets and blameless postmortems are examined directly, which makes it useful revision even if you never touch GCP.
LFCS	Medium	Worth it if your Linux is self-taught and patchy. The triage table in \$5 is the actual bar.
HashiCorp Terraform Associate	Medium	Cheap, quick, and evidences the infrastructure-as-code requirement.
Prometheus Certified Associate (PCA)	Medium	Narrow but precisely on target for the alerting and SLO parts of this role.

On certifications, from someone who has hired

Certificates get you past screening. They do not get you the offer. Given a choice between spending eight weeks on a certification and eight weeks building Beacon, build Beacon — then take a certification later if the market you are in expects one. In the interview room, a candidate who can open a repository and narrate a real incident on a system they built outperforms a candidate with three certifications and no artefacts, every time.

Keep these open while you work

- [Google SRE books](#) — all free online. The canon.
- [OpenTelemetry docs](#) · [semantic conventions](#) · [collector](#)
- [Grafana docs](#) · [Prometheus docs](#) · [ClickHouse docs](#)
- [PagerDuty incident response](#) · [public postmortem collection](#)
- [OpenGitOps principles](#) · [Argo CD](#) · [Flux](#)
- [OpenTelemetry Demo](#) — your load generator and reference implementation.
- [SRE resources hub](#) · [How They SRE](#) · [SREcon talks](#)
- [Brendan Gregg on Linux performance](#) — the reference for the triage half of the job.

One last thing

The strongest signal you can send in this interview is that you think of the platform as a product with users, and of reliability as something you negotiate rather than something you promise. Everything in this guide is downstream of that idea. Build Beacon, break it, write about it honestly, and you will have more to talk about than the interview has time for.

Site Reliability Engineer (Observability Platform) — Interview Learning Project. Prepared as a self-directed study programme for candidates. Independent study material; not affiliated with, endorsed by, or sponsored by any of the vendors or projects referenced.

OpenTelemetry, Prometheus, Kubernetes, Argo and Flux are projects of the Cloud Native Computing Foundation and the Linux Foundation. Grafana and Grafana Alloy are trademarks of Grafana Labs. ClickHouse is a trademark of ClickHouse, Inc. Cribl, Datadog, BigPanda, Redis, Apache Flink, Microsoft Azure, Amazon Web Services and Google Cloud are trademarks of their respective owners. All trademarks are the property of their respective owners and are referenced here for identification purposes only.

Tool versions, product features, certification names and syllabuses change. Verify current details against the official documentation linked throughout before relying on any specific version, setting or exam detail.